

École Polytechnique Fédérale de Lausanne

Thermal properties of MOFs from molecular dynamics

Master Thesis

Lucas De Almeida Kloss

November 13, 2026

Acknowledgments

I would like to thank Professor Michele Ceriotti for giving me the opportunity to do my master's thesis in his group. I had previously learned about molecular dynamics in some of my courses, but had never had the opportunity to apply it in a project. I had also previously taken courses on machine learning, but again had never had the opportunity to apply it in a real project. When I discovered the COSMO group on the EPFL website, I was immediately interested in the work they were doing and knew I wanted to do my master's thesis there. I have gained a lot of experience in numerical simulations, algorithms, machine learning and software development during this project, which will be very useful for my future career.

I would also like to thank Qianjun Xu for his help and guidance throughout the project, and for always being available to answer my questions. The discussions we had were very helpful in resolving the problems I encountered and saving time during the simulations. I would also like to thank Guillaume Fraux for explaining certain concepts when I was contributing to parts of the C++ API of the Metatomic and Metatensor libraries. I wanted to get better at programming and he gave me the opportunity to do so by letting me contribute and by reviewing my code.

AI usage disclosure

AI tools were used during this project mainly for code generation and syntax correction. The models used throughout the project were OpenAI's ChatGPT 5.6 Luna, 5.6 Terra, 5.6 Sol, 6 Astra and 6.1 Sol.

Contents

1 Introduction **3**

2 Theory **4**
- 2.1 Quantum Mechanics . . . 4
- 2.2 Density Functional Theory . . . 5
- 2.3 Molecular dynamics . . . 6
- 2.4 Heat Capacity . . . 8
- 2.5 Point Edge Transformer . . . 10

3 Methods **13**
- 3.1 MOFs . . . 13
- 3.2 Guest molecules . . . 13
- 3.3 PET-MAD and PET-SOL . . . 14
- 3.4 NPT simulation . . . 15
- 3.5 Hybrid C_P . . . 15
- 3.6 Uncertainty quantification . . . 16

4 Results **19**
- 4.1 MOF-5 . . . 19
- 4.2 MOF-303 . . . 23
- 4.3 ZIF-8 . . . 25

5 Conclusion **27**

References **28**

A Transformers and attention **32**
- A.1 Tokenization and embedding . . . 32
- A.2 Attention mechanism . . . 33
- A.3 Multi-head attention . . . 34

B Sparse automatic differentiation **35**
- B.1 The Jacobian, JVPs, and VJPs . . . 35
- B.2 Recovering multiple columns with a single evaluation . . . 36
- B.3 Graph coloring . . . 36
- B.4 Decompression . . . 37
- B.5 The Hessian and HVPs . . . 38
- B.6 ASD in PET . . . 39
- B.7 Hessian truncation . . . 40

C Temperature and pressure control **41**
- C.1 Thermostat . . . 41
- C.2 Barostat . . . 42

1 Introduction

Metal-organic frameworks (MOFs) are porous crystalline materials built from metal nodes and organic linkers, as their name suggests. These highly porous materials have a large internal surface area and can adsorb a wide variety of molecules, making them useful for applications such as gas storage [1]. As these materials adsorb and release gas molecules, they also exchange heat with their surroundings. The heat capacity influences the temperature changes during adsorption and desorption and the energy required to regenerate the adsorbent [25]. Therefore, studying their heat capacity is important for understanding these processes. The heat capacity of a material is defined as the amount of energy required to raise the temperature of the material by one degree. In 2019, Kapil et al. highlighted the lack of heat-capacity data for most MOFs and of studies on the influence of adsorbed guest molecules [25]. Covering all the potential gas molecules that can be adsorbed in the pores and wide temperature ranges presents a substantial challenge.

As a result, it would be useful to develop an accurate computational method to predict the heat capacity of MOFs. An efficient prediction method could help screen MOFs for favorable thermal properties [25]. However, because these materials and their adsorbed molecules form large and complex systems, there are many challenges when trying to model them accurately and efficiently. For empty MOFs, where only the solid framework is present, the heat capacity can be computed using the harmonic approximation, which is relatively inexpensive and can be accurate, as demonstrated for MOF-5 [25]. The problem arises when inserting guest molecules into the pores of the MOF. These molecules can move around more and interact with the framework as well as the other guest molecules. These interactions are anharmonic, which makes the harmonic approximation less accurate [25]. Further complicating matters, quantum effects can also play a role in these systems, especially at low temperatures. Classical molecular dynamics (MD) captures anharmonic motion but neglects nuclear quantum effects. Path integral molecular dynamics can account for both, although at a substantially higher computational cost [25]. For flexible, guest-loaded MOFs, allowing the simulation cell to respond to pressure can be important because constant-volume and constant-pressure properties can differ [25]. When running simulations that capture both quantum and anharmonic effects over a wide temperature range and at different loadings, the computational cost can become very high.

Capturing both quantum and anharmonic effects at an affordable computational cost is the central challenge addressed by Kapil et al. [25]. They used and evaluated a hybrid approach for MOF-5 at different CH_4 loadings and temperatures, combining classical MD with a harmonic correction to account for quantum effects. Our first goal in this project is to try to reproduce their results using PET [36], a machine-learned interatomic potential (MLIP). Once we have validated the approach, we will then extend it to other MOFs (MOF-303 and ZIF-8) and guest molecules (CO_2 and H_2O). Additionally, we will use a method proposed by Langer et al. [47] that uses automatic differentiation and the sparsity of the Hessian matrix to compute the harmonic heat capacity at a lower computational cost.

2 Theory

2.1 Quantum Mechanics

To better understand the methods used in this work, we briefly review some important concepts in quantum mechanics. These provide the basis for understanding the Born–Oppenheimer approximation, which defines the potential energy surface used in molecular dynamics simulations, and density functional theory, which is used to generate the reference energies, forces, and stresses for the machine-learned potentials.

2.1.1 Schrödinger's equation

In (non-relativistic) quantum mechanics, the fundamental equation that describes a quantum system is the Schrödinger equation [4]. Let $\hat{\mathcal{H}}$ be a Hermitian operator called the Hamiltonian, and let $|\Psi\rangle$ be a vector in a Hilbert space called a state vector. For a time-independent Hamiltonian, the time-independent Schrödinger equation describes energy eigenstates and can be written as

$$\hat{\mathcal{H}}|\Psi_n\rangle = E_n|\Psi_n\rangle, \tag{2.1}$$

where the eigenvalues E_n are real numbers and correspond to the energies of the states $|\Psi_n\rangle$. More generally, the time evolution of a state $|\Psi(t)\rangle$ is governed by the time-dependent Schrödinger equation, where the Hamiltonian $\hat{\mathcal{H}}(t)$ may also depend on time:

$$i\hbar\frac{\partial}{\partial t}|\Psi(t)\rangle = \hat{\mathcal{H}}(t)|\Psi(t)\rangle, \tag{2.2}$$

where $\hbar$ is the reduced Planck constant and i is the imaginary unit. The position representation is a way of expressing the state of a quantum system in terms of position. For instance, consider the Hermitian position operator $\hat{\boldsymbol{x}}$. Its eigenstates satisfy $\hat{\boldsymbol{x}}|\boldsymbol{x}\rangle = \boldsymbol{x}|\boldsymbol{x}\rangle$, where $\boldsymbol{x}$ is a real-valued position vector corresponding to the position of the particle. If we project $|\Psi(t)\rangle$ onto $|\boldsymbol{x}\rangle$, we obtain the wavefunction $\Psi(\boldsymbol{x}, t) = \langle\boldsymbol{x}|\Psi(t)\rangle$, which is a complex-valued function of position and time. In this representation, the momentum operator $\hat{\boldsymbol{p}}$ is also a Hermitian operator and is given by $\hat{\boldsymbol{p}} = -i\hbar\nabla_{\boldsymbol{x}}$. If we write $\hat{\mathcal{H}} = \hat{\boldsymbol{p}}^2/2m + \hat{V}$ in the position representation, we have $\hat{\mathcal{H}}(\boldsymbol{x}) = -\frac{\hbar^2}{2m}\nabla^2_{\boldsymbol{x}} + V(\boldsymbol{x})$. The Schrödinger equation in the position representation is then

$$i\hbar\frac{\partial\Psi(\boldsymbol{x}, t)}{\partial t} = \left[-\frac{\hbar^2}{2m}\nabla^2_{\boldsymbol{x}} + V(\boldsymbol{x})\right]\Psi(\boldsymbol{x}, t). \tag{2.3}$$

2.1.2 The Born–Oppenheimer approximation

Let us now consider a system of N_e electrons and N_n nuclei with coordinates $\boldsymbol{r}$ and $\boldsymbol{R}$, respectively. The Hamiltonian of this system in the position representation can be written as

$$\hat{\mathcal{H}}(\boldsymbol{r}, \boldsymbol{R}) = \hat{T}_\mathrm{e} + \hat{T}_\mathrm{n} + \hat{V}_\mathrm{ee}(\boldsymbol{r}) + \hat{V}_\mathrm{en}(\boldsymbol{r}; \boldsymbol{R}) + \hat{V}_\mathrm{nn}(\boldsymbol{R}), \tag{2.4}$$

where $\hat{T}_\mathrm{e}$ and $\hat{T}_\mathrm{n}$ are the kinetic energy operators for the electrons and nuclei, and $\hat{V}_\mathrm{ee}$, $\hat{V}_\mathrm{en}$, and $\hat{V}_\mathrm{nn}$ are the electron–electron, electron–nuclear, and nuclear–nuclear interaction potentials. Because nuclei are much heavier than electrons, we can approximately decouple the electronic and nuclear motions using the Born–Oppenheimer approximation [5]. We first separate the electronic

part of the Hamiltonian $\hat{\mathcal{H}}_{\mathrm{el}}(\boldsymbol{R}) = \hat{T}_{\mathrm{e}} + \hat{V}_{\mathrm{ee}} + \hat{V}_{\mathrm{en}}(\boldsymbol{R})$ from the nuclear part of the Hamiltonian $\hat{\mathcal{H}}_{\mathrm{nuc}} = \hat{T}_{\mathrm{n}} + \hat{V}_{\mathrm{nn}}(\boldsymbol{R})$. The approximation neglects terms arising from nuclear derivatives of the electronic wavefunction. We then solve the electronic Schrödinger equation by treating $\boldsymbol{R}$ as a fixed parameter:

$$\left[-\sum_a \frac{\hbar^2}{2m_{\mathrm{e}}}\nabla^2_{\boldsymbol{r}_a} + V_{\mathrm{ee}}(\boldsymbol{r}) + V_{\mathrm{en}}(\boldsymbol{r};\boldsymbol{R})\right]\psi_0(\boldsymbol{r};\boldsymbol{R}) = \varepsilon_0(\boldsymbol{R})\psi_0(\boldsymbol{r};\boldsymbol{R}). \tag{2.5}$$

By solving this equation for fixed nuclear coordinates $\boldsymbol{R}$, we obtain the electronic ground-state wavefunction $\psi_0(\boldsymbol{r};\boldsymbol{R})$ and its ground-state energy $\varepsilon_0(\boldsymbol{R})$. The potential energy surface for nuclear motion is then given by $U(\boldsymbol{R}) = \varepsilon_0(\boldsymbol{R}) + V_{\mathrm{nn}}(\boldsymbol{R})$. The nuclear Schrödinger equation is then given by

$$\left[-\sum_i \frac{\hbar^2}{2M_i}\nabla^2_{\boldsymbol{R}_i} + U(\boldsymbol{R})\right]\chi_\nu(\boldsymbol{R}) = E_\nu\chi_\nu(\boldsymbol{R}). \tag{2.6}$$

Solving this equation yields the nuclear wavefunction $\chi_\nu(\boldsymbol{R})$ and its energy E_ν. The total wavefunction of the system can then be approximated as a product of the electronic and nuclear wavefunctions

$$\Psi_\nu(\boldsymbol{r},\boldsymbol{R}) \approx \psi_0(\boldsymbol{r};\boldsymbol{R})\chi_\nu(\boldsymbol{R}). \tag{2.7}$$

2.2 Density Functional Theory

Within the Born–Oppenheimer approximation, the electronic ground-state energy $\varepsilon_0(\boldsymbol{R})$ must be determined for fixed nuclear coordinates $\boldsymbol{R}$. Solving Eq. (2.5) directly is computationally demanding because the many-electron wavefunction depends on all electronic coordinates. Density functional theory (DFT) instead uses the electron density $n(\boldsymbol{r})$, a function of only three spatial coordinates [11, 12].

2.2.1 The Hohenberg–Kohn theorems

The Hohenberg–Kohn (HK) theorems establish that the ground-state density determines the external potential, up to an additive constant, and hence the electronic Hamiltonian. They also provide a variational principle: the ground-state density minimizes the electronic energy functional at fixed electron number [11]:

$$\varepsilon_0(\boldsymbol{R}) = \min_{n \text{ admissible}} \left\{F[n] + \int v_{\mathrm{ext}}(\boldsymbol{r};\boldsymbol{R})n(\boldsymbol{r})\,\mathrm{d}\boldsymbol{r}\right\}, \qquad \int n(\boldsymbol{r})\,\mathrm{d}\boldsymbol{r} = N_{\mathrm{e}}. \tag{2.8}$$

Here v_{ext} is the external potential acting on an electron due to the nuclei, and the universal functional $F[n]$ contains the electronic kinetic and electron–electron interaction energies. The theorems establish this density formulation but do not give a practical expression for $F[n]$.

2.2.2 The Kohn–Sham equations

The HK theorems do not themselves provide a practical procedure for finding the ground-state density. The Kohn–Sham (KS) approach introduces a system of noninteracting electrons with the same ground-state density $n(\boldsymbol{r})$ as the interacting system [12]:

$$n(\boldsymbol{r}) = \sum_a f_a |\phi_a(\boldsymbol{r})|^2, \qquad \sum_a f_a = N_\mathrm{e}, \tag{2.9}$$

where $\phi_a(\boldsymbol{r})$ are the KS orbitals and f_a are their occupation numbers. The universal functional is written as $F[n] = T_\mathrm{s}[n] + E_\mathrm{H}[n] + E_\mathrm{xc}[n]$, where T_s is the noninteracting kinetic energy, E_H is the classical electrostatic energy of the electron density, and E_xc contains the remaining interaction and kinetic-energy contributions. Minimizing the energy with respect to the orbitals gives

$$\left[-\frac{\hbar^2}{2m_\mathrm{e}}\nabla^2 + v_\mathrm{ext}(\boldsymbol{r};\boldsymbol{R}) + v_\mathrm{H}[n](\boldsymbol{r}) + v_\mathrm{xc}[n](\boldsymbol{r})\right]\phi_a(\boldsymbol{r}) = \epsilon_a \phi_a(\boldsymbol{r}), \tag{2.10}$$

with $v_\mathrm{H} = \delta E_\mathrm{H}/\delta n$ and $v_\mathrm{xc} = \delta E_\mathrm{xc}/\delta n$. Because these potentials depend on the density, the equations are solved self-consistently by alternating orbital and density updates until convergence is reached [12]. A key challenge in obtaining accurate DFT results is choosing an appropriate approximation for the exchange-correlation functional $E_\mathrm{xc}[n]$.

2.3 Molecular dynamics

The potential $U(\boldsymbol{R}^N)$ appearing in the nuclear Schrödinger equation (2.6) is the Born–Oppenheimer potential energy surface. To describe the quantum evolution of the nuclei, we would solve the corresponding time-dependent Schrödinger equation. In classical molecular dynamics (MD), nuclear motion is instead approximated using classical mechanics, neglecting nuclear quantum effects. Let $\boldsymbol{R}^N = (\boldsymbol{R}_1, \ldots, \boldsymbol{R}_N)$ and $\boldsymbol{P}^N = (\boldsymbol{P}_1, \ldots, \boldsymbol{P}_N)$ be the positions and momenta of the N nuclei in the system. The classical Hamiltonian is then given by

$$\mathcal{H}(\boldsymbol{R}^N, \boldsymbol{P}^N) = \sum_{i=1}^{N} \frac{|\boldsymbol{P}_i|^2}{2M_i} + U(\boldsymbol{R}^N), \tag{2.11}$$

Given the potential energy surface and adopting the classical approximation for nuclear motion, Hamilton's equations give the equations of motion for the nuclei:

$$\dot{\boldsymbol{R}}_i = \frac{\partial \mathcal{H}}{\partial \boldsymbol{P}_i} = \frac{\boldsymbol{P}_i}{M_i}, \qquad \dot{\boldsymbol{P}}_i = -\frac{\partial \mathcal{H}}{\partial \boldsymbol{R}_i} = -\nabla_{\boldsymbol{R}_i} U(\boldsymbol{R}^N). \tag{2.12}$$

Because these $6N$ coupled ordinary differential equations generally cannot be solved analytically, we use numerical methods to integrate them and obtain the trajectories of the nuclei. Runge–Kutta methods are a popular family of iterative methods for solving ordinary differential equations. However, in molecular dynamics, the most commonly used method is the velocity Verlet algorithm [2]. The velocity Verlet algorithm is a symplectic integrator that preserves the symplectic structure of phase space and generally exhibits small energy fluctuations over long timescales when a suitable time step is used [6]. By writing the velocities as $\boldsymbol{V}_i(t) = \dot{\boldsymbol{R}}_i(t)$ and accelerations as $\boldsymbol{A}_i(t) = \dot{\boldsymbol{P}}_i(t)/M_i$, we can update the positions of the particles using

$$\boldsymbol{R}_i(t+\Delta t) = \boldsymbol{R}_i(t) + \boldsymbol{V}_i(t)\Delta t + \frac{1}{2}\boldsymbol{A}_i(t)\Delta t^2, \tag{2.13}$$

Using the updated positions, we can calculate $\boldsymbol{A}_i(t+\Delta t)$ and update the velocities using

$$\boldsymbol{V}_i(t+\Delta t) = \boldsymbol{V}_i(t) + \frac{1}{2}\left[\boldsymbol{A}_i(t) + \boldsymbol{A}_i(t+\Delta t)\right]\Delta t. \tag{2.14}$$

For simulations in the NVE ensemble, with fixed particle number N, volume V, and energy E, we can use the velocity Verlet algorithm to integrate the equations of motion. However, this project uses the NPT ensemble, with fixed N and imposed pressure P_{ext} and temperature T, to model laboratory conditions. We therefore use statistical mechanics to describe the appropriate ensemble.

2.3.1 Statistical Mechanics

The goal of statistical mechanics is to relate the microscopic properties of a system to its macroscopic behavior. In the context of molecular dynamics, we are interested in the properties of the NPT ensemble, which is characterized by a constant number of particles N, imposed external pressure P_{ext}, and imposed temperature T [7]. The quantity of interest in this case is the enthalpy, which we evaluate as $H = E + P_{\text{ext}}V$, where E is the internal energy and V is the volume. The microscopic internal energy is given by the Hamiltonian $\mathcal{H}(\boldsymbol{R}^N, \boldsymbol{P}^N)$, so the microscopic enthalpy is

$$H(\boldsymbol{R}^N, \boldsymbol{P}^N; V) = \sum_{i=1}^{N} \frac{|\boldsymbol{P}_i|^2}{2M_i} + U(\boldsymbol{R}^N) + P_{\text{ext}}V. \tag{2.15}$$

In the NPT ensemble, the internal energy, enthalpy, and instantaneous temperature and pressure fluctuate as the system exchanges heat and work with its surroundings. A macroscopic observable is obtained as the ensemble average of the corresponding microscopic observable $A(\boldsymbol{R}^N, \boldsymbol{P}^N)$ [25]:

$$\langle A\rangle = \frac{1}{\mathcal{Z}_{\text{NPT}}}\int_0^\infty \mathrm{d}V \int_{\text{cell}(V)} \mathrm{d}^{3N}R \int \mathrm{d}^{3N}P A(\boldsymbol{R}^N, \boldsymbol{P}^N) e^{-\beta[\mathcal{H}(\boldsymbol{R}^N,\boldsymbol{P}^N)+P_{\text{ext}}V]}, \tag{2.16}$$

Here $\beta = (k_{\text{B}}T)^{-1}$, where k_{B} is the Boltzmann constant, and $\mathcal{Z}_{\text{NPT}}$ is the same integral with $A = 1$, ensuring normalization. The particles are confined to a periodic simulation cell of volume V and shape $\boldsymbol{h}$. Additionally, we assume the ergodic hypothesis, which states that the infinite-time average of an observable along an equilibrium trajectory equals its ensemble average [7]. The ensemble average in Eq. (2.16) can therefore be expressed as

$$\langle A\rangle = \lim_{\tau\to\infty}\frac{1}{\tau}\int_0^\tau A\big(\boldsymbol{R}^N(t), \boldsymbol{P}^N(t)\big)\,\mathrm{d}t. \tag{2.17}$$

In practice, simulations have a finite duration and must be long enough for the system to reach equilibrium and explore the relevant phase space. After discarding the equilibration period, we approximate the integral using M equally spaced trajectory frames:

$$\langle A\rangle \approx \frac{1}{M}\sum_{j=1}^{M} A\big(\boldsymbol{R}^N(t_j), \boldsymbol{P}^N(t_j)\big). \tag{2.18}$$

The temperature and the pressure are controlled by a thermostat and a barostat to sample the NPT ensemble. The corresponding equations and their implementations are described in Appendix C.

2.4 Heat Capacity

The heat capacity of a material measures the energy required to raise its temperature by 1 K. In the context of MOFs, the heat capacity is an important property for understanding their thermal behavior and stability [25]. The heat capacity can be defined at constant volume C_V or at constant pressure C_P. In this project, we focus on the constant-pressure heat capacity C_P, which is relevant to applications under imposed pressure. There are different ways to compute it depending on the level of theory used to describe the system. We present the methods used in this project below.

2.4.1 C_P

As discussed earlier, the heat capacity is defined as the amount of energy required to raise the temperature of a system by 1 K. In the NPT ensemble, C_P is the temperature derivative of the average enthalpy [7]:

$$C_P(T) = \left(\frac{\partial \langle H \rangle}{\partial T}\right)_{N,P_{\text{ext}}}, \qquad \langle H \rangle = \langle E + P_{\text{ext}} V \rangle. \tag{2.19}$$

The average enthalpy $\langle H \rangle$ is obtained from the MD trajectory as a time average, as shown in Eq. (2.18). If we perform MD simulations at different temperatures, we can estimate C_P using a centered finite difference [25]:

$$C_P(T) \simeq \frac{\langle H \rangle_{T+\Delta T} - \langle H \rangle_{T-\Delta T}}{2\Delta T}. \tag{2.20}$$

2.4.2 Dulong–Petit law

The heat capacity of a solid at constant volume can be estimated using the Dulong–Petit law [9]. In a solid, the atoms are arranged in a lattice and can vibrate around their equilibrium positions. The potential energy of the system can be approximated as a quadratic function of the atomic displacements. In a simplified model of independent Cartesian oscillators, the Hamiltonian is a sum of quadratic terms in the displacements $\boldsymbol{u}^N$ and momenta:

$$\mathcal{H}(\boldsymbol{u}^N, \boldsymbol{P}^N) = \sum_{i=1}^{N} \frac{|\boldsymbol{P}_i|^2}{2M_i} + \frac{1}{2}\sum_{i=1}^{N}(k_{i,x}u_{i,x}^2 + k_{i,y}u_{i,y}^2 + k_{i,z}u_{i,z}^2), \tag{2.21}$$

where $k_{i,x}$, $k_{i,y}$, and $k_{i,z}$ are the force constants for the i-th atom in the x, y, and z directions, respectively. Using the equipartition theorem, each quadratic term contributes $k_{\text{B}}T/2$ to the average energy [7]. The average energy is therefore $\langle E \rangle_{\text{har}} = 3Nk_{\text{B}}T$. At constant volume, the heat capacity is then given by

$$C_{V,\text{cl}}^{\text{har}} = \left(\frac{\partial \langle E \rangle_{\text{har}}}{\partial T}\right)_{N,V} = 3Nk_{\text{B}}. \tag{2.22}$$

Within the classical harmonic approximation, the constant-volume heat capacity is independent of temperature. This law does not hold well at lower temperatures, where quantum effects become important.

2.4.3 Quantum harmonic heat capacity from the Hessian

The harmonic approximation can also be used to compute the quantum harmonic heat capacity of a solid. Consider N atoms displaced by $\boldsymbol{u}$ from their equilibrium configuration $\boldsymbol{R}_0$. The harmonic approximation gives

$$U(\boldsymbol{R}_0+\boldsymbol{u}) \simeq U_0 + \frac{1}{2}\boldsymbol{u}^{\mathsf{T}}\boldsymbol{H}\boldsymbol{u}, \qquad H_{i\alpha,j\beta} = \left.\frac{\partial^2 U}{\partial R_{i\alpha}\partial R_{j\beta}}\right|_{\boldsymbol{R}_0}. \tag{2.23}$$

Using the diagonal mass matrix $\boldsymbol{M}$, which contains the masses of the atoms, we can define the mass-weighted Hessian $\boldsymbol{D}$ and its eigenvectors $\boldsymbol{e}_s$ as

$$\boldsymbol{D} = \boldsymbol{M}^{-1/2}\boldsymbol{H}\boldsymbol{M}^{-1/2}, \qquad \boldsymbol{D}\boldsymbol{e}_s = \omega_s^2\boldsymbol{e}_s. \tag{2.24}$$

By finding the eigenvalues ω_s^2 of $\boldsymbol{D}$, we can write the quantum harmonic Hamiltonian in terms of the mass-weighted normal-mode coordinates $\hat{Q}_s$ and momenta $\hat{P}_s$:

$$\hat{\mathcal{H}}_{\text{har}} = U_0 + \sum_s \left(\frac{\hat{P}_s^2}{2} + \frac{1}{2}\omega_s^2\hat{Q}_s^2\right). \tag{2.25}$$

Because the Hamiltonian is a sum of independent quantum harmonic oscillators, we can solve the Schrödinger equation (2.1) for each mode independently. Their energy levels are given by $E_{s,n} = \hbar\omega_s(n+1/2)$, where n is a non-negative integer [4]. Using the canonical partition function

$$Z = e^{-\beta U_0}\prod_s\sum_{n=0}^{\infty} e^{-\beta\hbar\omega_s(n+1/2)} = e^{-\beta U_0}\prod_s \frac{e^{-\beta\hbar\omega_s/2}}{1-e^{-\beta\hbar\omega_s}}, \tag{2.26}$$

we can compute the average energy of the system as

$$\langle E\rangle_{\text{har}} = -\frac{\partial \ln Z}{\partial\beta} = U_0 + \sum_s \hbar\omega_s\left(\frac{1}{2} + \frac{1}{e^{\beta\hbar\omega_s}-1}\right). \tag{2.27}$$

The heat capacity at constant volume is then given by [9, 10]:

$$C_{V,\text{qn}}^{\text{har}}(T) = \left(\frac{\partial\langle E\rangle_{\text{har}}}{\partial T}\right)_{N,V} = \sum_s k_{\text{B}}x_s^2\frac{e^{x_s}}{(e^{x_s}-1)^2}, \qquad x_s = \frac{\hbar\omega_s}{k_{\text{B}}T}. \tag{2.28}$$

Thus, the eigenvalues of the mass-weighted Hessian determine the quantum harmonic heat capacity. As $T\to\infty$, $C_{V,\text{qn}}^{\text{har}} \to N_{\text{vib}}k_{\text{B}}$, where N_{vib} is the number of included vibrational modes; counting $3N$ modes recovers the Dulong–Petit limit $3Nk_{\text{B}}$.

2.5 Point Edge Transformer

A machine-learned interatomic potential (MLIP) approximates a system's energy from atomic positions and species, providing an inexpensive alternative to direct electronic-structure calculations within its range of applicability. The MLIPs used in this project are based on the Point Edge Transformer (PET) architecture [36], which combines local attention with message passing between atomic environments.

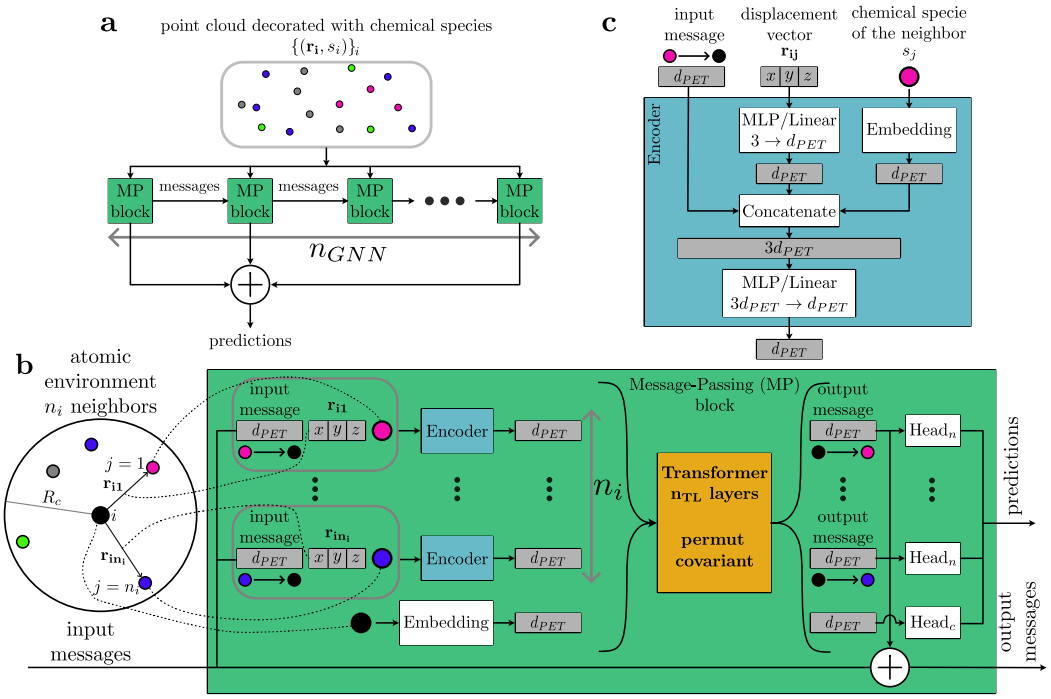


Figure 1: Point Edge Transformer architecture, reproduced from Ref. [36]. Each message-passing block encodes the local geometry, species, and incoming messages, processes them with a transformer, and produces energy contributions and outgoing messages.

Model overview For each atom i, PET defines a local environment $A_i = \{(\boldsymbol{r}_{ij}, s_j) \mid r_{ij} < R_c\}$, where $\boldsymbol{r}_{ij}$ is the position of neighbor j relative to i, s_j is its species, and R_c is the cutoff radius. The same operations are applied to every environment within each message-passing (MP) block. Each block produces energy contributions and messages that carry information to neighboring environments in the next block. Repeating this process through n_{GNN} blocks allows the prediction to incorporate information beyond the initial cutoff neighborhood (Figure 1a). The final energy combines contributions from all atoms and blocks.

Encoding and transformer processing At block k, the species and relative positions are converted into learned feature vectors, or embeddings, of width d_{PET}. Separate maps $E_c^{(k)}$ and $E_n^{(k)}$ encode the central and neighbor species, while $E_r^{(k)}$ encodes the Cartesian displacement using a linear layer and a SiLU activation. The neighbor embedding and incoming message are combined into a token [36, Appendix A]:

$$t_j^{(k)} = M^{(k)}\left[\text{concatenate}\left(\tilde{x}_{ij}^{(k-1)}, E_n^{(k)}(s_j), E_r^{(k)}(\boldsymbol{r}_{ij})\right)\right]. \tag{2.29}$$

Here $M^{(k)}$ is a multilayer perceptron that returns a token of width d_{PET}, and $\tilde{x}_{ij}^{(k-1)}$ is the

message sent from j to i in the previous block. The first block omits the message input. Together with the central token $E_c^{(k)}(s_i)$, an environment with m neighbors supplies $m+1$ tokens to the transformer.

The tokens pass through n_{TL} transformer layers, combining multi-head self-attention with feedforward operations, residual connections, and layer normalization [30]. Attention lets each token use information from the other atoms in its environment, so its output reflects the surrounding geometry. Reordering the neighbor tokens produces the same reordering of their outputs. Transformer depth increases processing within one environment, whereas MP depth extends communication between environments. The attention mechanism is described in Appendix A.

Smooth cutoff A neighbor's influence must vanish smoothly as it approaches R_c to avoid discontinuities in the predicted energy. This is particularly important here because forces and Hessians require first and second energy derivatives. PET uses the following cutoff function [36, Appendix F.1]:

$$f_c(r \mid R_c, \Delta_{R_c}) = \begin{cases} 1, & r \le R_c - \Delta_{R_c}, \\ \frac{1}{2}\left[\tanh\left(\frac{1}{x+1} + \frac{1}{x-1}\right) + 1\right], & R_c - \Delta_{R_c} < r < R_c, \\ 0, & r \ge R_c, \end{cases}$$
$$x = 2\frac{r - R_c + 0.5\Delta_{R_c}}{\Delta_{R_c}}. \tag{2.30}$$

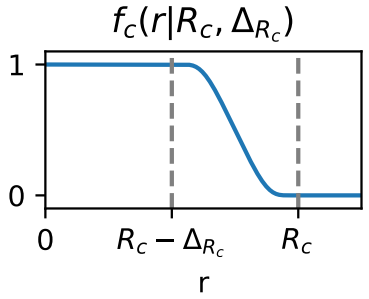


Figure 2: Smooth cutoff function from Eq. (2.30) with $R_c = 5.0$ Å and $\Delta_{R_c} = 0.5$ Å. The neighbor's weight decreases from one to zero over the interval $[R_c - \Delta_{R_c}, R_c]$.

The cutoff modifies the attention weights and their normalization, so a neighbor disappears smoothly from the information used by the remaining tokens. For one attention head,

$$\alpha_{ab}^{\mathrm{cut}} = \frac{\alpha_{ab} c_b}{\sum_c \alpha_{ac} c_c}, \qquad \boldsymbol{y}_a = \sum_b \alpha_{ab}^{\mathrm{cut}} \boldsymbol{v}_b, \tag{2.31}$$

where α_{ab} are the ordinary attention weights from Eq. (A.2), $\boldsymbol{v}_b$ is a value vector, and $c_b = f_c(r_{ib} \mid R_c, \Delta_{R_c})$ for a neighbor, with $c_b = 1$ for the central token.

Messages and energy prediction Each transformer returns a central output $x_i^{(k)}$ and one output $x_{ji}^{(k)}$ per neighbor. The neighbor outputs are added to previous messages through a residual update and passed to the corresponding neighboring environments for the next MP block [36, Appendix A]. Learned readouts $H_c^{(k)}$ and $H_n^{(k)}$ convert the central and neighbor outputs into scalar energy contributions. The neighbor terms are also weighted by the cutoff to remove their direct energy contribution smoothly:

$$y_i^{\mathrm{bond}\,(k)} = \sum_{j \in A_i} H_n^{(k)}(x_{ji}^{(k)}) f_c(r_{ij} \mid R_c, \Delta_{R_c}). \tag{2.32}$$

The paper also permits a cutoff-weighted average instead of this sum. Including the fitted atomic baseline $E_{\mathrm{self}} = \sum_i b_{s_i}$, the total energy can be written as

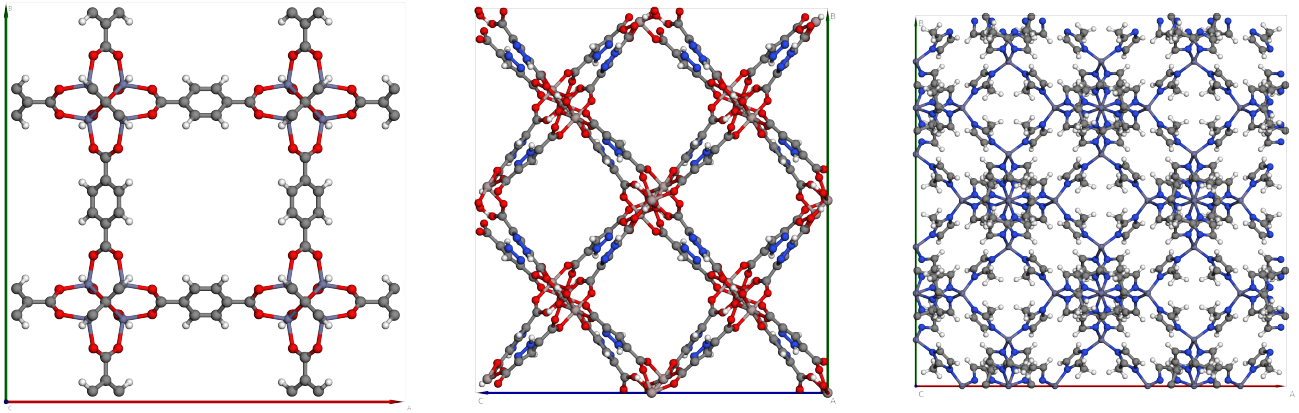
$$\tilde{E} = E_{\text{self}} + \sum_{k=1}^{n_{\text{GNN}}} \sum_{i} \left[H_c^{(k)}(x_i^{(k)}) + y_i^{\text{bond}\,(k)} \right]. \tag{2.33}$$

This expression combines the readout and baseline described in Ref. [36]; $\tilde{E}$ is the learned potential U_θ used in this report. The baseline is fitted to species counts before training and has no Cartesian derivatives at fixed composition. The central and edge terms are learned parts of the total energy, rather than uniquely defined physical atomic or bond energies. Forces and Hessians follow by differentiating the total prediction, including its dependence on all local environments and messages.

3 Methods

3.1 MOFs

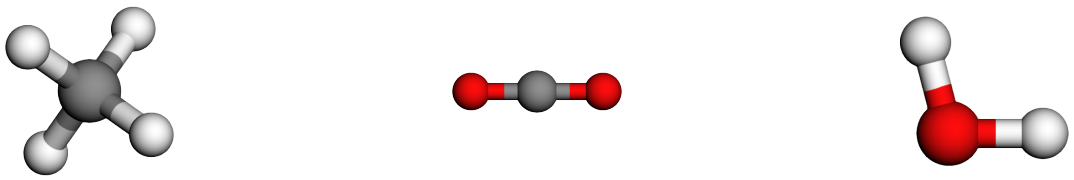
The methane comparison uses the conventional cubic MOF-5 cell, with initial lattice parameter 25.866 Å and composition $C_{192}H_{96}O_{104}Zn_{32}$ (424 host atoms). Its Zn_4O nodes are connected by benzene-1,4-dicarboxylate linkers [25]. Adding 50, 100, or 150 methane molecules gives 674, 924, or 1174 atoms, respectively. Periodic boundary conditions apply in all three directions. MOF-303, Al(OH)(PZDC), contains aluminium–oxygen rods connected by 1H-pyrazole-3,5-dicarboxylate linkers [13]; its $2 \times 2 \times 2$ host supercell contains 1024 atoms. ZIF-8, $Zn(mIm)_2$, contains zinc centres joined by 2-methylimidazolate linkers in a sodalite-type network [14]; its $2 \times 2 \times 2$ host supercell contains 2208 atoms. Both PET models are compared for methane-loaded MOF-5. PET-MAD is also used for MOF-5 with 50 CO_2 or H_2O molecules, MOF-303 with 50 molecules of each guest, and ZIF-8 with 50 CH_4 or 54 CO_2 molecules.



(a) MOF-5 (b) MOF-303 (c) ZIF-8

Figure 3: Empty MOFs used in this project and rendered with Chemiscope [49]. Atoms are colored by species, including C (gray), H (white), O (red), and Zn (blue). MOF-5 has formula $C_{192}H_{96}O_{104}Zn_{32}$, MOF-303 has formula $Al_{64}C_{320}H_{192}N_{128}O_{320}$, and ZIF-8 has formula $C_{768}H_{960}N_{384}Zn_{96}$. MOF-5 contains 424 atoms; the MOF-303 and ZIF-8 $2 \times 2 \times 2$ supercells contain 1024 and 2208 atoms, respectively.

3.2 Guest molecules

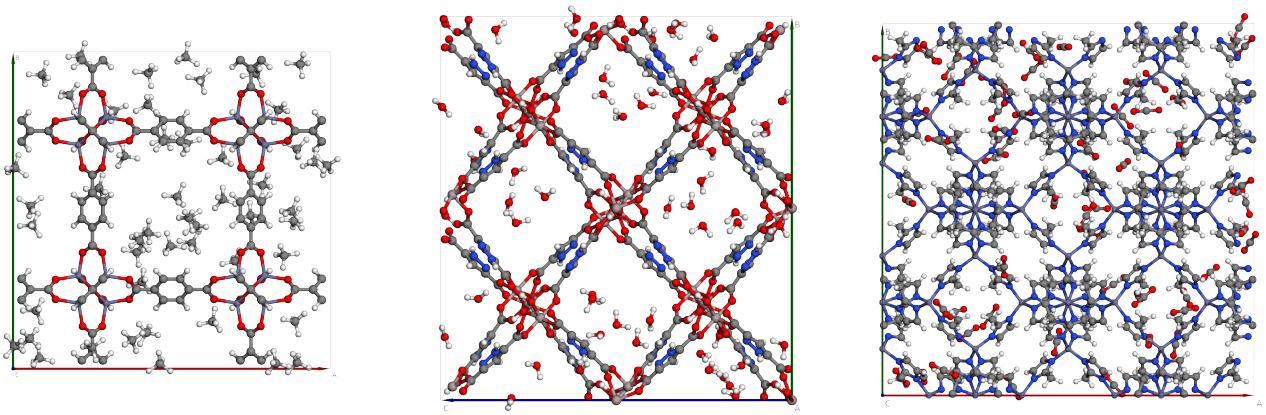


(a) CH_4 (b) CO_2 (c) H_2O

Figure 4: Guest molecules used in this project and rendered with Chemiscope [49]. H_2O is the only polar molecule, while CH_4 and CO_2 are non-polar.

The guest molecules are inserted sequentially at random positions and orientations within the MOF cell. Placements are accepted only when guest atoms remain at least 1.5 Å from the

framework and other molecules, accounting for periodic images. If sequential insertion becomes blocked, partial or complete repacking is attempted within a finite placement budget. Accepted structures are checked for periodic separation before use. Figure 5 shows representative starting configurations with guests inside the same periodic host cells as in Figure 3, before relaxation.



(a) MOF-5 + 50 CH_4 (b) MOF-303 + 50 H_2O (c) ZIF-8 + 54 CO_2

Figure 5: Guest-loaded MOFs rendered with Chemiscope [49], using the same viewing directions and element colors as the empty frameworks. MOF-5 contains 50 methane molecules (674 atoms in total), the MOF-303 $2 \times 2 \times 2$ supercell contains 50 water molecules (1174 atoms), and the ZIF-8 $2 \times 2 \times 2$ supercell contains 54 carbon dioxide molecules (2370 atoms). These are insertion configurations before relaxation.

After insertion and any initial relaxation, velocities are sampled from a Maxwell–Boltzmann distribution at the target temperature [6]:

$$P(\boldsymbol{v}_i) = \left(\frac{M_i}{2\pi k_\mathrm{B} T}\right)^{3/2} \exp\left(-\frac{M_i |\boldsymbol{v}_i|^2}{2 k_\mathrm{B} T}\right). \tag{3.1}$$

3.3 PET-MAD and PET-SOL

Two pretrained PET models are used in this work, PET-MAD and PET-SOL. They share the PET architecture described in Section 2.5 but differ in their training data. PET-MAD-1.5 uses MAD-1.5, spanning 102 elements and including molecules, clusters, crystals, and low-dimensional structures, with reference energies, forces, and periodic stresses computed using all-electron r^2SCAN DFT [33]. PET-SOL incorporates additional MAD-SOL solvation data covering 17 solvents, 78 cations, and 35 anions alongside the parent dataset, using the same reference level. The PET-SOL remains unpublished at the time of writing. The small variants, PET-MAD-1.5-s and PET-SOL-s, are used to reduce the MD cost. Their shared settings are summarized in Table 1.

Setting	Value
R_c, Δ_{R_c}	8.0 Å, 0.5 Å
n_{GNN}, n_{TL}	3, 1
d_{PET}	256
Attention heads	8
Adaptive neighbor setting	40
Transformer feed-forward width	512
Node feature width, `d_node`	1024

Table 1: Shared architecture settings of the supplied PET-MAD and PET-SOL models.

3.4 NPT simulation

Classical NPT simulations are performed with LAMMPS [3], using its metatomic interface to evaluate PET energies, forces, and stresses [48]. The cell lengths and angles fluctuate under hydrostatic pressure, using the LAMMPS Nosé–Hoover-chain and Martyna–Tobias–Klein/Shinoda equations [42, 38, 39, 17]. The particle thermostat and barostat each use a chain of length three. One trajectory is run independently at each temperature for each model and loading; Table 2 summarizes the settings.

Parameter	Value
T, ΔT	175–425 K, 25 K
P_{ext}	1 bar
τ_T, τ_P	100 fs, 1000 fs
t, Δt	500 ps, 0.5 fs
t_{eq}	100 ps
Frame interval	0.05 ps

Table 2: Parameters used in the NPT simulations.

Each 500 ps trajectory contains one million integration steps; the first 200,000 steps (100 ps) are discarded as equilibration. Saving every 100 steps (0.05 ps) leaves 8000 production frames. The reported range is 200–400 K; additional runs at 175 and 425 K provide centered finite differences at its endpoints through Eq. (2.20). Each MD run uses one NVIDIA Tesla V100 GPU with 32 GB of memory.

3.5 Hybrid C_P

The harmonic approximation can describe the empty-framework heat capacity, as demonstrated for MOF-5, but guest motion introduces anharmonicity that it cannot capture [25]. Following Kapil et al., the loaded-system estimate combines classical NPT sampling with a harmonic quantum correction:

$$C_P^{\mathrm{hyb}}(T) \simeq C_{P,\mathrm{cl}}^{\mathrm{anh}}(T) + C_{V,\mathrm{qn}}^{\mathrm{har}}(T) - C_{V,\mathrm{cl}}^{\mathrm{har}}(T). \tag{3.2}$$

The classical term is obtained from Eq. (2.20), and the quantum term from Eq. (2.28). Both harmonic terms use the same retained loaded-system modes, so $C_{V,\mathrm{cl}}^{\mathrm{har}} = N_{\mathrm{vib}} k_{\mathrm{B}}$. This construction retains classical anharmonicity while approximating the quantum correction harmonically. It neglects quantum anharmonic effects and assumes that the quantum change in $C_P - C_V$ is small.

Empty frameworks are relaxed directly and evaluated harmonically as separate references; no empty-framework MD is performed.

3.5.1 Computing the frequencies

For each loaded system, the final configuration of the 400 K trajectory is relaxed at fixed cell with the central model using ASE FIRE [34]. The target maximum force is 0.002 eV/Å, except for the two MOF-5 100-methane calculations, which use 0.01 eV/Å. Recorded limits range from 10,000 to 20,000 optimizer steps. The resulting central and LLPR spectra are reused at all reported temperatures; the temperature dependence enters the harmonic occupation factors rather than a new Hessian calculation at every temperature.

The Hessians are evaluated using truncated automatic sparse differentiation [47] and ASDEX [35]. The calculations use float64 precision, three graph hops, and a chunk size of one. This method is explained in more detail in Appendix B.

3.6 Uncertainty quantification

When using a machine learning model to predict quantities, there are many sources of uncertainty. The model may not have been trained on data that is representative of the system being studied. It may not capture all relevant physics, which depends on the first principles methods used when doing DFT calculations. If we wanted to have better uncertainty estimates, we would need to have the model trained on different datasets and use these models to predict the same quantity. We could then have a better idea on the uncertainty of the model predictions. Additionally, we could also build complete different models with different architectures and train them with different datasets. Combining the results from each model to have a final prediction and uncertainty estimate is what we call ensemble learning. In traditional machine learning, ensemble learning is a technique that combines the predictions of multiple models to improve overall performance and provide more robust uncertainty estimates. In practice, ensemble learning is widely used in various machine learning tasks, including regression, classification, and forecasting. One of the most efficient models are tree-based models, such as Random Forests and Gradient Boosting Machines, which combine the predictions of multiple decision trees to improve accuracy and reduce overfitting.

3.6.1 Last-layer prediction rigidity

Training multiple models, especially in the case of MLIPs, is an expensive task. Instead of training multiple models, we can use the last-layer prediction rigidity (LLPR) [43] method to estimate the uncertainty of a single model. The idea is that instead of training multiple models, we take the model we are using for the simulation and we create new models by just changing its last layer. The models with the different last layers could be considered as different models that were trained on different datasets. This method is cheap and efficient, and it has been shown to provide good uncertainty estimates for MLIPs [43]. For central energy $\bar{U} = \boldsymbol{w}_0^{\mathrm{T}} \boldsymbol{f}(\boldsymbol{x})$ and covariance-set features $\boldsymbol{F}$,

$$\begin{aligned} \sigma^2_{\mathrm{LLPR}}(\boldsymbol{x}) = \alpha^2 \boldsymbol{f}(\boldsymbol{x})^{\mathrm{T}} \\ \times \left(\boldsymbol{F}^{\mathrm{T}}\boldsymbol{F} + \lambda \boldsymbol{I}\right)^{-1} \boldsymbol{f}(\boldsymbol{x}). \end{aligned} \tag{3.3}$$

The regularizer λ stabilizes weakly constrained directions; α is calibrated against energy residuals on a disjoint reference-labeled set using the base potential's electronic-structure definition. Predictions cannot calibrate their own error. The exported `energy_uncertainty` is an energy standard deviation. Each new model is generated by sampling the last-layer weight vector $\boldsymbol{w}_i$ from the following distribution

$$\boldsymbol{w}_i \sim \mathcal{N}\left[\boldsymbol{w}_0, \alpha^2 \left(\boldsymbol{F}^{\mathrm{T}}\boldsymbol{F} + \lambda \boldsymbol{I}\right)^{-1}\right], \tag{3.4}$$

The model is then created by multiplying the sampled weight vector with the weights from all the previous layers of the model, which are kept frozen.

$$U^{(i)}(\boldsymbol{x}) = \boldsymbol{w}_i^{\mathrm{T}} \boldsymbol{f}(\boldsymbol{x}) \tag{3.5}$$

The advantage of this method is that it is not only cheap to generate the models, but also we do not need to rerun the simulations with them. The model used to run the simulation can be considered to be the central model and the mean of all the other models generated by sampling the last layer is equal to the central model. That means that we can use the same trajectory generated by the central model to evaluate the quantities with the other models, which will be treated as uncertainties.

3.6.2 Cumulative expansion approximation

We can then compute the difference between the potential of the members and the central model as well as the enthalpy of each member.

$$\Delta U_t^{(i)}(T) = U_t^{(i)}(T) - \bar{U}_t(T), \tag{3.6}$$

$$H_t^{(i)}(T) = K_t + U_t^{(i)}(T) + PV_t, \tag{3.7}$$

In particular, if we want to calculate the ensemble average of the enthalpy as a function of time, we can use the following equation

$$\left\langle H^{(i)}(T) \right\rangle = \frac{\sum_t e^{-\beta \Delta U_t^{(i)}(T)} H_t^{(i)}}{\sum_t e^{-\beta \Delta U_t^{(i)}(T)}}, \tag{3.8}$$

It is done using the Boltzmann factors of the energy differences between the central model and the members. When the energy differences $\Delta U_t^{(i)}$ are small, the direct reweighting can be approximated by a first-order cumulant expansion approximation (CEA):

$$\begin{aligned} \left\langle H^{(i)}(T) \right\rangle \approx \left\langle H^{(i)}(T) \right\rangle_{\bar{U}} - \beta \operatorname{cov}_{\bar{U}} \left(H^{(i)}(T), \Delta U^{(i)}(T)\right), \\ \beta = (k_{\mathrm{B}} T)^{-1}. \end{aligned} \tag{3.9}$$

The CEA is a useful approximation when the energy differences are small, but it is not exact. Direct reweighting remains a diagnostic: $N_{\mathrm{eff}} = 1/\sum_t \widetilde{w}_t^2$ should remain adequate, and $\mathrm{var}_t(\beta \Delta U_t^{(i)}) \ll 1$ is needed for controlled first-order CEA. Applying Eq. (2.20) to each member's CEA mean enthalpies gives $C_{P,\mathrm{cl}}^{(i)}(T)$. We can then simply calculate the uncertainty at each temperature as the standard deviation of the heat capacities of all the members:

$$\sigma_{C_{P,\mathrm{cl,anh}}}(T) = \left\{ \frac{1}{M-1} \sum_{i=1}^{M} \left[C_{P,\mathrm{cl}}^{(i)}(T) - \overline{C}_{P,\mathrm{cl}}(T) \right]^2 \right\}^{1/2}. \tag{3.10}$$

This term corresponds to the classical anharmonic heat capacity uncertainty. To compute the uncertainty of the quantum harmonic heat capacity, we need first to select the last frame in the simulation, relax the structure and then compute the Hessian. We then calculate the eigenvalues of the Hessian matrix, which correspond to the vibrational frequencies of the system. We will compute the Hessian for each model and its eigenvalues, and then we will compute the quantum harmonic heat capacity for each model using Eq. (2.28). Finally, we can compute the uncertainty of the quantum harmonic heat capacity as the standard deviation of the heat capacities of all the models:

$$\sigma_{C_{V,\mathrm{qn,har}}}(T) = \left\{ \frac{1}{M-1} \sum_{i=1}^{M} \left[C_{V,\mathrm{har}}^{(i)}(T) - \overline{C}_{V,\mathrm{har}}(T) \right]^2 \right\}^{1/2}. \tag{3.11}$$

The total uncertainty of the hybrid heat capacity is then obtained by combining the uncertainties of the classical anharmonic and quantum harmonic heat capacities in quadrature:

$$\sigma_{C_P}(T) = \sqrt{\sigma^2_{C_{P,\mathrm{cl,anh}}}(T) + \sigma^2_{C_{V,\mathrm{qn,har}}}(T)}. \tag{3.12}$$

4 Results

The loaded-system curves are hybrid c_P estimates, while empty-framework references are harmonic c_V values. The reported loaded spectra contain imaginary modes that were excluded from the harmonic sums, so these results remain exploratory. The shaded uncertainties do not include all systematic errors from spectral filtering, incomplete equilibration, or the CEA.

4.1 MOF-5

4.1.1 CH_4

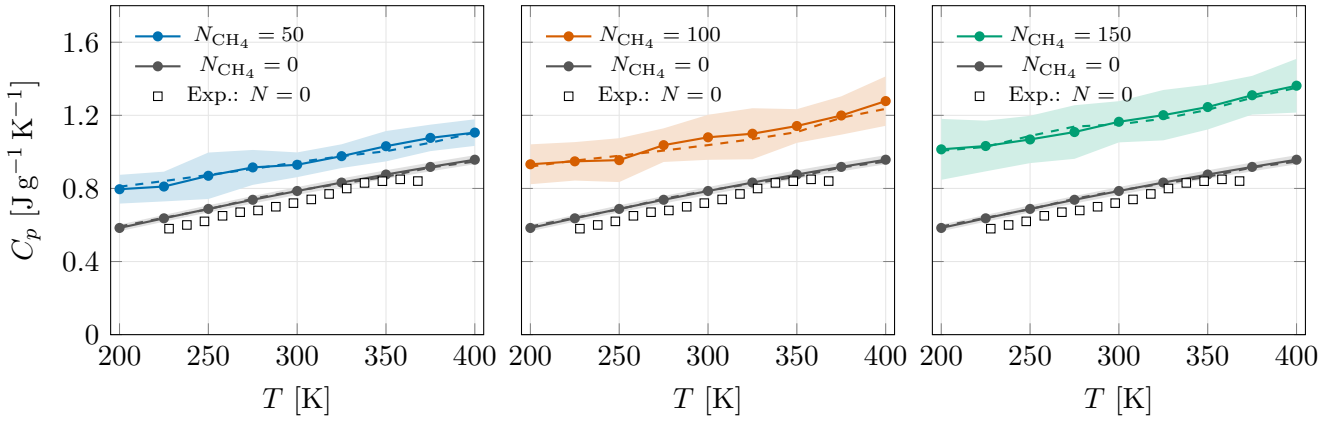


Figure 6: MOF-5 C_P with 50, 100, and 150 CH_4 molecules per cell using PET-MAD. Dashed lines show the results of Kapil *et al.* [25]. Open squares show experimental empty-MOF-5 data from Ming *et al.* [27]. Shading denotes standard uncertainty; details are given in Section 3.6.

Kapil *et al.* [25] used force fields derived from first-principles data with QuickFF, a force-field parameterization tool for MOFs [37]. PET-MAD gives very similar heat capacities, with differences of at most 0.043 J/g/K at 300 K. This agreement shows that a general-purpose potential can closely reproduce the reference heat capacities for the methane-loaded MOF-5 systems studied here. Its broader applicability motivates testing other frameworks, although relative computational efficiency was not benchmarked against the QuickFF force fields.

The reference curves were obtained using PIMD, which includes nuclear quantum effects as well as anharmonic motion [25]. Combining classical PET-MAD simulations with the harmonic quantum correction reproduces these curves closely without performing PIMD. This supports the hybrid approximation for the tested systems, but does not establish its accuracy for every host and guest.

A major difference from the work of Kapil *et al.* concerns the uncertainty calculation: the present estimates include both MD and model uncertainty. Figure 6 shows the uncertainty bands obtained using 64 committee members. The three loadings are presented in separate panels to avoid overlapping bands.

The empty framework has much narrower uncertainty bands than the loaded systems. Its heat capacity requires no MD simulation, so its bands reflect only the LLPR spread of the harmonic heat capacities. The loaded-system bands also widen with increasing loading. This may reflect greater sensitivity to the guest–guest and guest–host interactions, but the energy spread alone

does not determine the heat-capacity uncertainty, which also depends on enthalpy reweighting and the vibrational spectra.

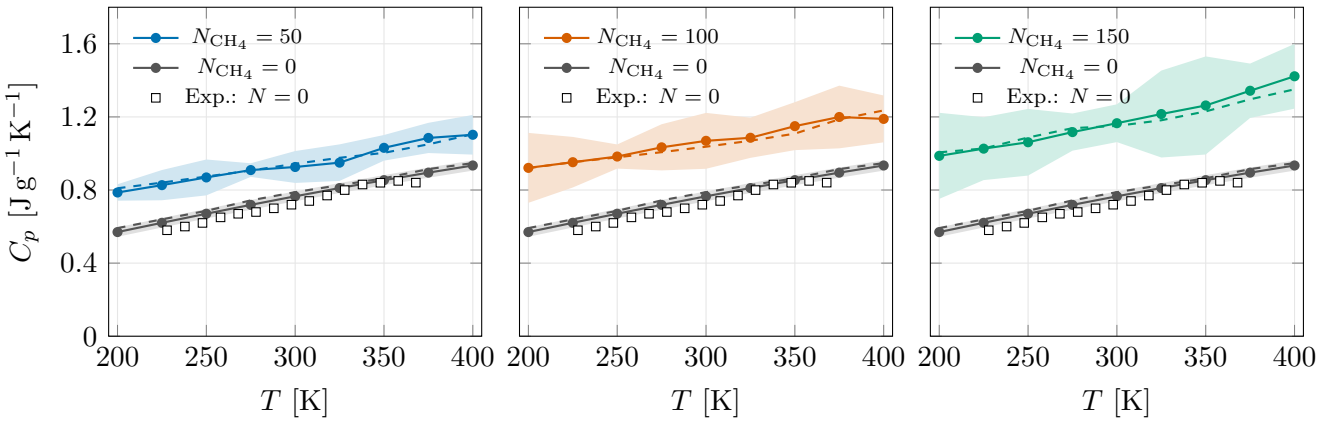


Figure 7: MOF-5 C_P with 50, 100, and 150 CH_4 molecules per cell using PET-SOL. Dashed lines show the results of Kapil *et al.* [25]. Open squares show experimental empty-MOF-5 data from Ming *et al.* [27]. Shading denotes standard uncertainty; details are given in Section 3.6.

PET-SOL also reproduces the QuickFF results accurately (Figure 7). At 300 K, the differences are at most 0.043 J/g/K. Despite its additional solvation training data, PET-SOL does not improve the overall agreement for methane-loaded MOF-5. Its overall deviation from the reference curves is slightly larger, although the comparison depends on loading.

The agreement of both potentials with the reference curves motivates extending classical MD and the hybrid method to other MOFs and guest molecules. To conserve computational resources, the remaining simulations use only PET-MAD, which gives smaller overall deviations in this comparison. Table 3 summarizes the deviations from the reference data.

Model	CH_4/cell	MAE	RMSE	Bias	max $\lvert e_i \rvert$
PET-MAD	50	0.0140	0.0178	0.0000	0.0300
PET-MAD	100	0.0256	0.0287	0.0194	0.0428
PET-MAD	150	0.0150	0.0165	0.0041	0.0299
PET-SOL	50	0.0171	0.0204	-0.0027	0.0348
PET-SOL	100	0.0201	0.0258	0.0097	0.0464
PET-SOL	150	0.0297	0.0353	0.0147	0.0714

Table 3: Deviations of the MOF-5/CH_4 hybrid heat capacities from the digitized QuickFF/PIMD reference of Kapil *et al.* [25]. Each row uses nine simulation temperatures from 200 to 400 K in 25 K steps; reference values are linearly interpolated to these temperatures without extrapolation. With $e_i = c_{P,i}^{\mathrm{sim}} - c_{P,i}^{\mathrm{ref}}$, the mean absolute error (MAE) is $\langle \lvert e_i \rvert \rangle$, the root mean square error (RMSE) is $\sqrt{\langle e_i^2 \rangle}$, the bias (mean error) is $\langle e_i \rangle$, and max $\lvert e_i \rvert$ is the largest absolute deviation. Positive bias indicates overestimation. All deviations are in J/g/K and compare central estimates without uncertainty weighting.

4.1.2 CO_2 and H_2O

No experimental heat-capacity dataset for CO_2-loaded MOF-5 is used here, and Kapil *et al.* [25] did not study this system. The calculated curve behaves qualitatively like the methane curve, but

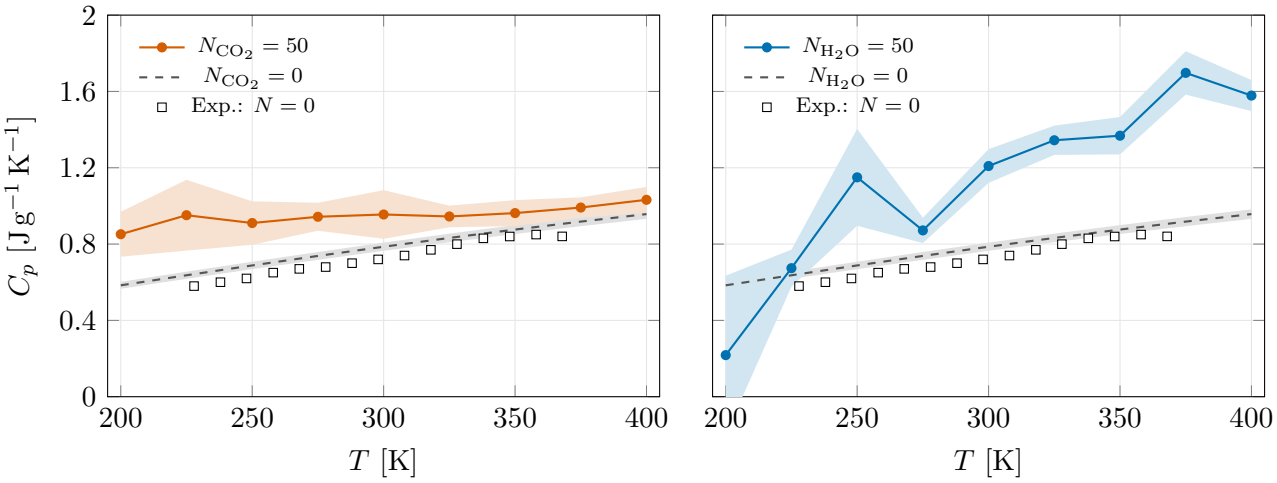


Figure 8: MOF-5 heat capacities with 50 CO_2 molecules (left) and 50 H_2O molecules (right) per cell using PET-MAD. Each panel shows the total hybrid c_P and the empty-framework harmonic c_V reference. Loaded curves use the host-plus-guest mass; empty references use the host mass. Open squares show experimental empty-MOF-5 data from Ming *et al.* [27]. Shading denotes standard uncertainty; details are given in Section 3.6. Both y-axes start at zero; uncertainty bands extending below zero are clipped at the axis boundary.

appears to approach a plateau and is very close to the empty-framework heat capacity at 400 K. These are mass-specific values, normalized by different total masses, so a loaded curve can lie below the empty reference without violating thermodynamics. The apparent plateau alone does not establish the behavior beyond the simulated temperature range.

Kapil *et al.* [25] averaged multiple MD simulations at each temperature to obtain a more accurate heat-capacity curve. Time constraints limited this project to one simulation per temperature, leaving room for improvement.

For MOF-5 with 50 H_2O molecules, the heat capacity at 200 K lies below the empty-framework reference. This ordering alone is not unphysical on a mass-specific basis, but the trajectory diagnostics below show that the low-temperature estimate is sensitive to equilibration.

The calculated heat capacity decreases substantially between 250 and 275 K, with a smaller change between 350 and 375 K. Such features could reflect changes in guest organization, but assigning them to melting or vaporization from the heat-capacity curve alone is not justified. Confined water at fixed loading need not follow the phase boundaries of bulk water. Longer trajectories and structural diagnostics are needed to distinguish a transition from incomplete equilibration. Experimental comparison would also help assess these results.

PET-SOL is a possible alternative because its training includes solvation-specific data. Time constraints prevented this comparison. The differences between water and the nonpolar guests motivate this test, but do not by themselves establish that PET-MAD is less suitable for polar guests in general.

Figure 9 presents the trajectory diagnostics at 200 K. Although the mean temperature is 200.10 K, the mean enthalpy decreases by 0.559 eV per cell from the 100–300 ps interval to the 300–500 ps interval, while the mean volume increases by 0.187%. The system therefore continues to relax during the measurement interval despite maintaining the target temperature.

Recomputing the hybrid heat capacity over the early and late intervals with the same harmonic quantum correction gives 0.363 J/g/K and 0.073 J/g/K, respectively, compared with 0.218 J/g/K

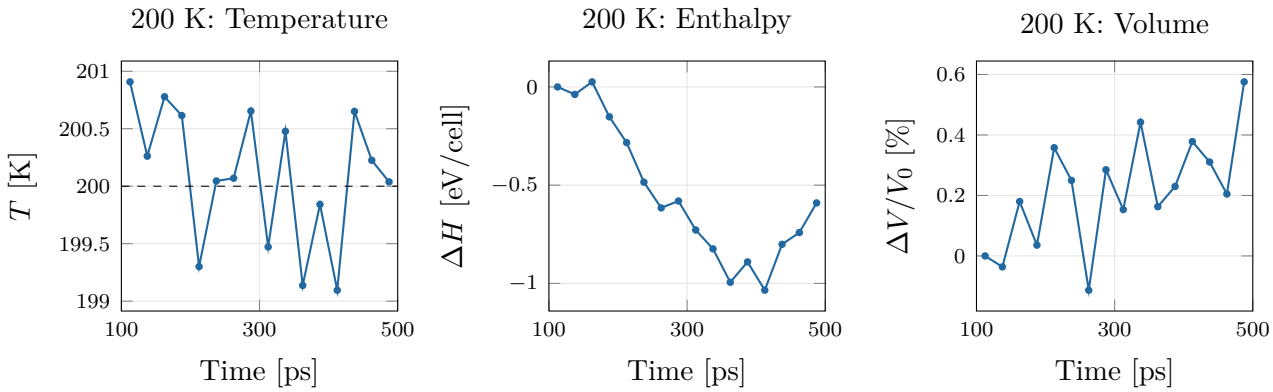


Figure 9: Trajectory diagnostics for MOF-5 with 50 H_2O molecules at 200 K. Points are nonoverlapping 25 ps block means over the 100–500 ps measurement interval. The dashed line marks the target temperature. Enthalpy and volume changes are measured relative to their first block means; V_0 is the first block mean volume. The blocks summarize one trajectory and are not independent replicas.

over the full interval. The derivative at 200 K depends on the mean enthalpies at 175 and 225 K, so this sensitivity concerns the equilibration of both the neighboring-temperature trajectories and the 200 K trajectory itself.

In nine frames sampled from the 200 K trajectory, O–H distances range from 0.915 to 1.091 Å. Each water hydrogen remains closest to its original oxygen, even when framework oxygens are included in the periodic search. These spot checks show no obvious dissociation in the sampled frames but cannot exclude events between samples. The enthalpy drift and sensitivity to the averaging window demonstrate a convergence limitation; they neither establish a phase transition nor fully explain the low heat capacity.

4.2 MOF-303

4.2.1 CH_4, CO_2, and H_2O

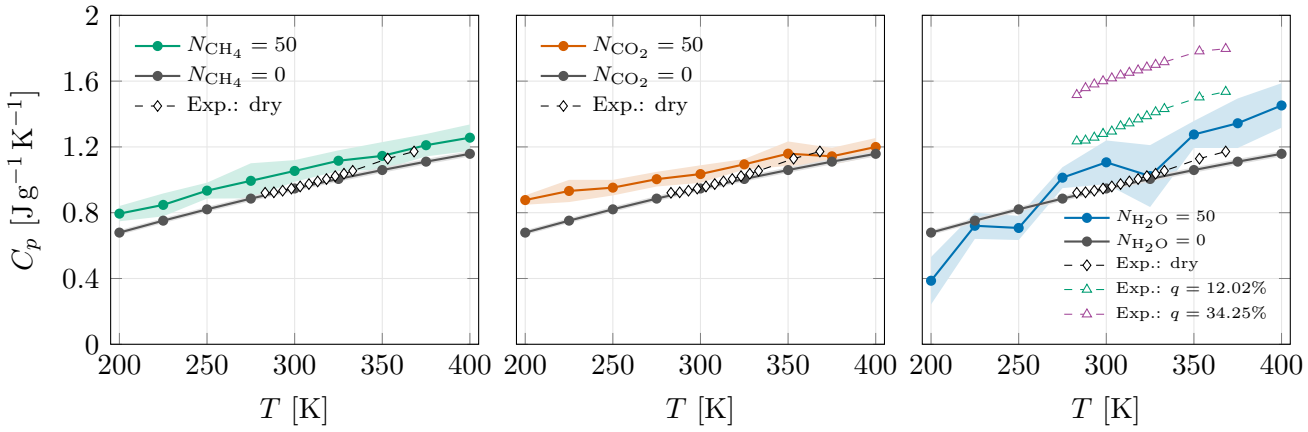


Figure 10: PET-MAD heat capacities for MOF-303 with 50 CH_4 (left), 50 CO_2 (middle), and 50 H_2O (right) molecules per $2 \times 2 \times 2$ supercell. Each panel shows the total hybrid c_P and empty-framework harmonic c_V reference. Loaded curves use the host-plus-guest mass; empty references use the host mass. The methane panel uses its campaign-specific empty Hessian. Shading denotes standard uncertainty. Dashed experimental curves connect dry-MOF-303 measurements in each panel and water loadings $q = 12.02\%$ and 34.25% in the right panel [26]; experimental uncertainties were not supplied. Here q is the fraction of saturated water loading, and the simulation corresponds to $q = 16.76\%$. Dry measurements refer to the empty framework. All y-axes start at zero.

Abu El-maaty *et al.* [26] report water loading as a fraction of the saturated loading, $q = m_{\text{water}}/m_{\text{water,sat}}$. Their dry and saturated sample masses of 7.3990 and 10.5358 mg give a saturated water-to-dry-framework mass ratio of 0.42395. In the simulation, 50 water molecules in the 1024-atom host give a water-to-host mass ratio of 0.07106, corresponding to $q = 0.16761$. This composition lies between the experimental samples with $q = 0.1202$ and $q = 0.3425$. Following the experimental trend with loading, the simulated heat capacity would be expected to lie between these curves, but instead falls below both. This is not an exactly composition-matched comparison: the simulation fixes the number of guests, while the experimental loading and sealed-sample conditions introduce additional differences.

As in MOF-5, the water-loaded heat capacity falls below the empty-framework reference at low temperatures and is not strictly monotonic. Neither feature is intrinsically unphysical, but the lack of stationarity shown below limits interpretation. Longer sampling is needed before attributing the discrepancy to the potential or quantum treatment. PIMD could test the neglected quantum anharmonic effects, and PET-SOL could test sensitivity to the potential.

Figure 11 shows the water-trajectory diagnostics at 200, 225, and 250 K, where the hybrid heat capacity lies below the empty-framework curve. The mean temperatures are 199.96, 224.91, and 249.84 K, respectively. Comparing the 300–500 ps interval with the 100–300 ps interval gives mean enthalpy changes of -0.553, -0.363, and -0.795 eV per supercell, accompanied by mean volume increases of 0.087%, 0.153%, and 0.318%. All three trajectories therefore continue to relax during the measurement interval despite maintaining their target temperatures.

Keeping the harmonic quantum correction fixed while using the early and late averaging intervals gives early/late hybrid heat capacities of $(0.358, 0.415)$, $(0.738, 0.703)$, and $(0.736, 0.679)$

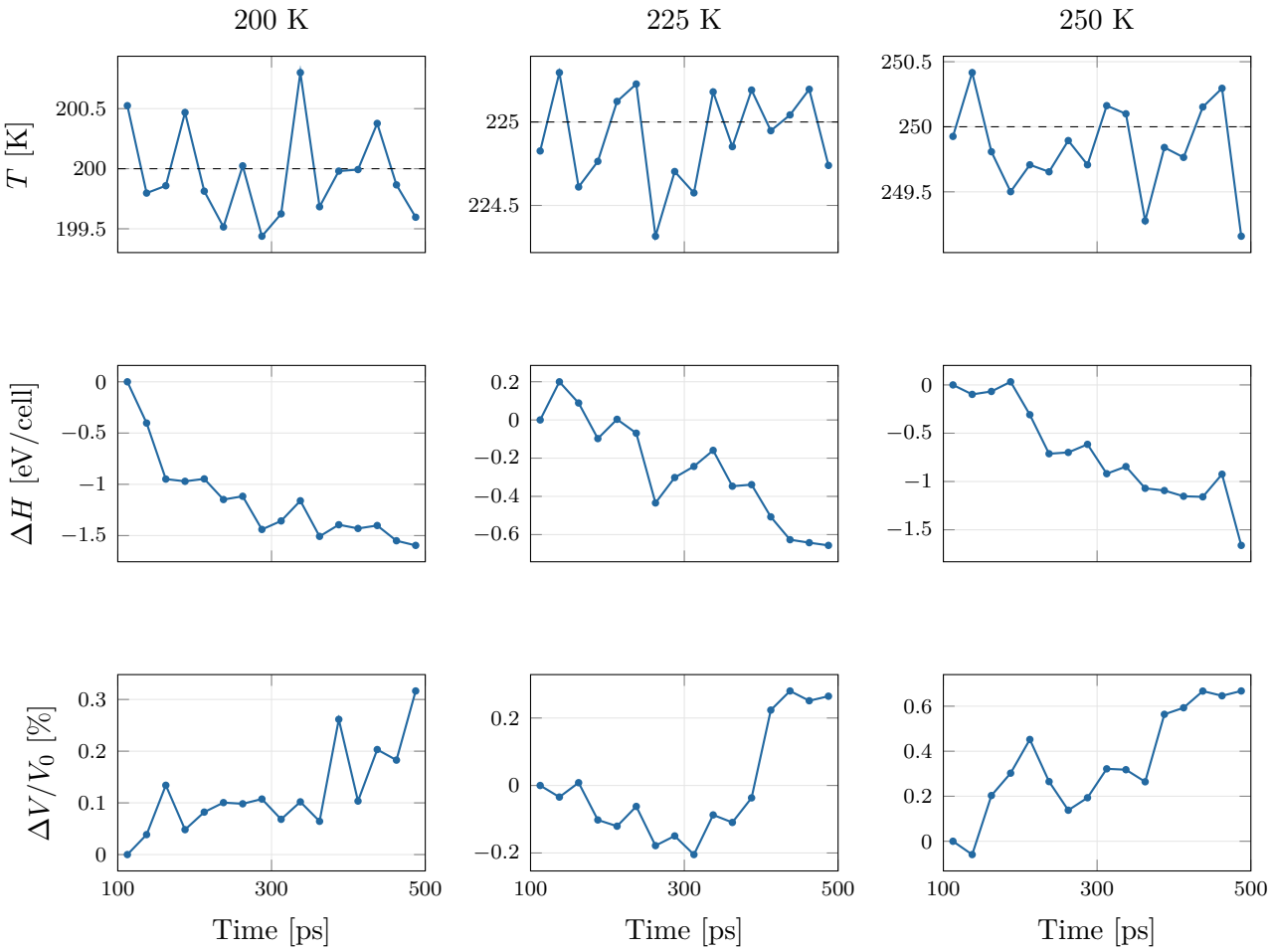


Figure 11: Trajectory diagnostics for MOF-303 with 50 H_2O molecules per $2 \times 2 \times 2$ supercell at 200, 225, and 250 K (left to right). Rows show temperature, enthalpy change, and relative volume change (top to bottom). Points are nonoverlapping 25 ps block means over 100–500 ps; dashed lines mark the target temperatures. Each trajectory's enthalpy and volume changes are referenced to its first block means. These diagnostics use one trajectory per temperature.

in J/g/K at 200, 225, and 250 K, respectively. The full-interval values are 0.386, 0.721, and 0.707 J/g/K. These estimates are correlated and do not constitute independent uncertainty bounds; each temperature derivative also depends on neighboring-temperature runs.

Structural spot checks of nine frames per trajectory give O–H distances between 0.905 and 1.067 Å, with each water hydrogen closest to its original oxygen. The diagnostics show that stationarity has not been reached, but do not demonstrate molecular breakup or fully explain the discrepancy with the empty-framework heat capacity.

4.3 ZIF-8

For CO_2-loaded ZIF-8 at 300 K, the classical contribution $c_{\mathrm{cl}} = 2.426$ and harmonic correction $\Delta c_{\mathrm{har}} = -1.460$ give $c_{\mathrm{hyb}} = 0.967 \pm 0.151\,\mathrm{J\,g^{-1}\,K^{-1}}$. The hybrid curve rises from 0.805 at 200 K to 1.200 J/g/K at 400 K (Figure 12). Comparisons with the other frameworks must account for differences in guest concentration and mass normalization.

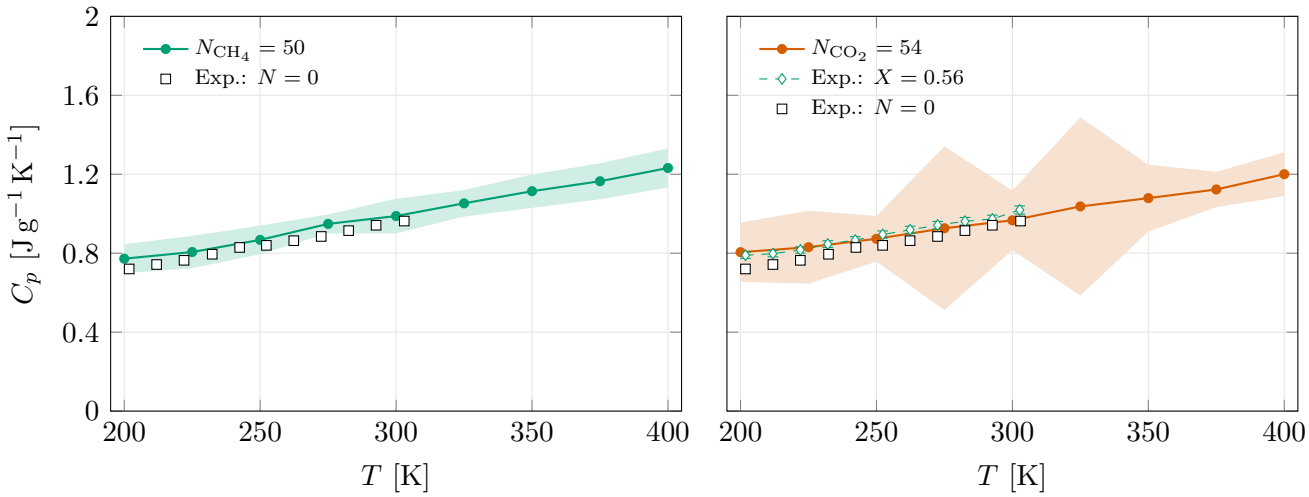


Figure 12: PET-MAD total hybrid heat capacities with standard uncertainty for 50 CH_4 (left) and 54 CO_2 (right) molecules in the ZIF-8 $2 \times 2 \times 2$ supercell. The CO_2 panel includes experimental measurements at $X = 0.56$ mol CO_2/mol ZIF-8 [29]; the simulation has $X = 0.5625$. The experimental molar values from Supporting Information Table 3 are divided by the loaded sample molar mass, 252.3 g/mol, to give the same total-sample mass basis as the simulation. Experimental error bars show 2% standard uncertainty in this temperature range; dashed lines connect measured points. The experiment reports sealed-cell C, whereas the simulation estimates C_P. Open squares show independent experimental empty-ZIF-8 (SOD) data [28], converted from molar units using 227.576 g/mol. The digitized empty-framework datasets do not include uncertainty values, so no error bars are inferred for them. Both y-axes start at zero.

The simulated heat capacities are numerically close to the experimental measurements over the overlapping temperature range. On the common 200–300 K grid, linear interpolation between neighboring experimental temperatures gives signed simulation differences ranging from approximately +2.2% to −4.0%. At 300 K, the interpolated experimental value is approximately 1.007 J/g/K, compared with the simulated $0.967 \pm 0.151\,\mathrm{J\,g^{-1}\,K^{-1}}$. The 2% experimental standard uncertainty corresponds to approximately 0.020 J/g/K at this temperature and does not include a separate interpolation uncertainty.

The measurement conditions qualify this numerical agreement. The experiment contains a fixed total amount of CO_2 in a sealed pan. Neglecting pan expansion, the authors describe the

measured heat capacity as approximately constant-volume for the entire system, including any free CO_2 outside the crystal [29]. The periodic NPT simulation contains all 54 CO_2 molecules inside the framework. Moreover, the initial experimental sealing pressure is not maintained throughout the temperature scan. The 200–400 K simulation curve also samples only the upper tail of the reported 70–220 K structural anomaly.

5 Conclusion

PET-MAD and PET-SOL, combined with classical MD and a harmonic quantum correction, closely reproduce the QuickFF/PIMD reference heat capacities for methane-loaded MOF-5. PET-MAD gives slightly smaller overall deviations and is used for the additional host–guest systems. This agreement supports the hybrid approach for the tested methane systems, rather than establishing accuracy for all MOFs and guests.

At room temperature, the MOF-5 heat capacities rise from about 0.93 to 1.16 J/g/K with methane loading. Harmonic quantization removes 59–63% of the classical term, while guest motion motivates retaining anharmonic MD. Additional PET-MAD estimates at 300 K are 0.955/1.209 for CO_2/H_2O in MOF-5, 1.035/1.106 for CO_2/H_2O in MOF-303, and 0.967 J/g/K for CO_2 in ZIF-8. The MOF-303/water estimate lies below the measured neighboring-loading curves, whereas ZIF-8/CO_2 is within about 4% of the matched-composition measurement. Loading, mass normalization, and sealed-cell versus constant-pressure conditions qualify these comparisons.

The water-loaded trajectories continue to drift, and all reported loaded spectra contain imaginary modes. Their exclusion and the limitations of CEA prevent interpreting the curves and uncertainty bands as fully converged predictions. Independent trajectories, stable spectra, and validated uncertainty propagation are priorities. PIMD or alternative potentials could then test the remaining physical approximations.

The hybrid method offers a practical route to heat-capacity estimates without full PIMD, but its computational advantage was not benchmarked here. A finer temperature grid could reduce finite-difference bias; at fixed enthalpy precision, however, a smaller ΔT increases the statistical noise in the derivative. It should therefore be tested together with longer simulations and independent replicas, rather than assumed to reduce the uncertainty automatically.

References

[1] H. Furukawa, K. E. Cordova, M. O'Keeffe, and O. M. Yaghi, *The Chemistry and Applications of Metal-Organic Frameworks*, Science **341**, 1230444 (2013), doi:10.1126/science.1230444.

[2] W. C. Swope, H. C. Andersen, P. H. Berens, and K. R. Wilson, *A computer simulation method for the calculation of equilibrium constants for the formation of physical clusters of molecules: Application to small water clusters*, Journal of Chemical Physics **76**, 637–649 (1982), doi:10.1063/1.442716.

[3] A. P. Thompson *et al.*, *LAMMPS—a flexible simulation tool for particle-based materials modeling at the atomic, meso, and continuum scales*, Computer Physics Communications **271**, 108171 (2022), doi:10.1016/j.cpc.2021.108171; LAMMPS website.

[4] C. Cohen-Tannoudji, B. Diu, and F. Laloë, *Quantum Mechanics, Volume I: Basic Concepts, Tools, and Applications*, 2nd ed., Wiley-VCH, Weinheim (2020), publisher edition.

[5] M. Born and R. Oppenheimer, *Zur Quantentheorie der Molekeln*, Annalen der Physik **84**, 457–484 (1927), doi:10.1002/andp.19273892002.

[6] M. P. Allen and D. J. Tildesley, *Computer Simulation of Liquids*, 2nd ed., Oxford University Press (2017), publisher edition.

[7] M. E. Tuckerman, *Statistical Mechanics: Theory and Molecular Simulation*, 2nd ed., Oxford University Press (2023), publisher edition.

[8] R. M. Martin, *Electronic Structure: Basic Theory and Practical Methods*, Cambridge University Press (2004), doi:10.1017/CBO9780511805769.

[9] N. W. Ashcroft and N. D. Mermin, *Solid State Physics*, Holt, Rinehart and Winston, New York (1976), Chaps. 22–23: Classical and quantum theory of the harmonic crystal.

[10] C. Kittel, *Introduction to Solid State Physics*, 8th ed., Wiley, Hoboken (2005), Chap. 5: Phonons II. Thermal Properties, publisher edition.

[11] P. Hohenberg and W. Kohn, *Inhomogeneous Electron Gas*, Physical Review **136**, B864–B871 (1964), doi:10.1103/PhysRev.136.B864.

[12] W. Kohn and L. J. Sham, *Self-Consistent Equations Including Exchange and Correlation Effects*, Physical Review **140**, A1133–A1138 (1965), doi:10.1103/PhysRev.140.A1133.

[13] N. Hanikel *et al.*, *Evolution of Water Structures in Metal-Organic Frameworks for Improved Atmospheric Water Harvesting*, Science **374**, 454–459 (2021), doi:10.1126/science.abj0890.

[14] C. L. Hobday *et al.*, *Understanding the Adsorption Process in ZIF-8 Using High Pressure Crystallography and Computational Modelling*, Nature Communications **9**, 1429 (2018), doi:10.1038/s41467-018-03878-6.

[15] C. Rowley, *Periodic Boundary Conditions in 2D* (2015), Wikimedia Commons, CC BY-SA 4.0.

[16] LAMMPS documentation, *compute pressure command*, online documentation (accessed 2 October 2026).

[17] LAMMPS documentation, *fix npt command*, online documentation (accessed 2 November 2026).

[18] LAMMPS developers, *Nosé–Hoover integrator implementation*, `FixNH::compute_scalar`, source code (accessed 15 October 2026).

[19] LAMMPS documentation, *min_style fire command*, online documentation (accessed 13 October 2026).

[20] H. Flyvbjerg and H. G. Petersen, *Error Estimates on Averages of Correlated Data*, Journal of Chemical Physics **91**, 461–466 (1989), doi:10.1063/1.457480.

[21] M. Ceriotti, M. Parrinello, T. E. Markland, and D. E. Manolopoulos, *Efficient Stochastic Thermostatting of Path Integral Molecular Dynamics*, Journal of Chemical Physics **133**, 124104 (2010), doi:10.1063/1.3489925.

[22] R. Diestel, *Graph Theory*, 6th ed., Graduate Texts in Mathematics **173**, Springer (2025), author's textbook site.

[23] A. H. Gebremedhin, A. Pothen, A. Tarafdar, and A. Walther, *Efficient Computation of Sparse Hessians Using Coloring and Automatic Differentiation*, INFORMS Journal on Computing **21**, 209–223 (2009), doi:10.1287/ijoc.1080.0286.

[24] A. Hill, *Graph Coloring*, ASDEX documentation, online documentation.

[25] V. Kapil *et al.*, *Modeling the Structural and Thermal Properties of Loaded Metal–Organic Frameworks: An Interplay of Quantum and Anharmonic Fluctuations*, Journal of Chemical Theory and Computation **15**, 3237–3249 (2019), doi:10.1021/acs.jctc.8b01297.

[26] A. E. Abu El-maaty, O. Saoudi, A. S. Abdelrazik, and R. Ben-Mansour, *Experimental investigation of specific heat and thermal stability of MOF-303 at different water vapor loading and various temperatures*, International Communications in Heat and Mass Transfer **165**, 109088 (2025), doi:10.1016/j.icheatmasstransfer.2025.109088.

[27] Y. Ming, J. Purewal, D. Liu, A. Sudik, C. Xu, J. Yang, M. Veenstra, K. Rhodes, R. Soltis, J. Warner, M. Gaab, U. Müller, and D. J. Siegel, *Thermophysical properties of MOF-5 powders*, Microporous and Mesoporous Materials **185**, 235–244 (2014), doi:10.1016/j.micromeso.2013.11.015.

[28] P. F. Rosen, J. J. Calvin, M. S. Dickson, A. D. Katsenis, T. Friščić, A. Navrotsky, N. L. Ross, A. I. Kolesnikov, and B. F. Woodfield, *Heat capacity and thermodynamic functions of crystalline forms of the metal-organic framework zinc 2-methylimidazolate, Zn(MeIm)$_2$*, The Journal of Chemical Thermodynamics **136**, 160–169 (2019), doi:10.1016/j.jct.2019.05.008.

[29] P. F. Rosen, M. S. Dickson, J. J. Calvin, N. L. Ross, T. Friščić, A. Navrotsky, and B. F. Woodfield, *Thermodynamic Evidence of Structural Transformations in CO_2-Loaded Metal–Organic Framework Zn(MeIm)$_2$ from Heat Capacity Measurements*, Journal of the American

Chemical Society **142**, 4833–4841 (2020), doi:10.1021/jacs.9b13883. Experimental values and uncertainties are from Supporting Information Table 3, doi:10.1021/jacs.9b13883.s001.

[30] A. Vaswani *et al.*, *Attention Is All You Need*, Advances in Neural Information Processing Systems **30** (2017), arXiv:1706.03762.

[31] T. B. Brown *et al.*, *Language Models are Few-Shot Learners*, Advances in Neural Information Processing Systems **33**, 1877–1901 (2020), arXiv:2005.14165.

[32] A. Mazitov *et al.*, *PET-MAD as a Lightweight Universal Interatomic Potential for Advanced Materials Modeling*, Nature Communications **16**, 10653 (2025), doi:10.1038/s41467-025-65662-7.

[33] C. Malosso *et al.*, *High-quality, high-information dataset for universal atomistic machine learning*, arXiv:2603.02089v1 (2026), arXiv:2603.02089v1.

[34] E. Bitzek, P. Koskinen, F. Gähler, M. Moseler, and P. Gumbsch, *Structural Relaxation Made Simple*, Physical Review Letters **97**, 170201 (2006), doi:10.1103/PhysRevLett.97.170201.

[35] ASDEX developers, *ASDEX: Automatic Sparse Differentiation in JAX*, software documentation, online documentation (accessed 13 November 2026).

[36] S. N. Pozdnyakov and M. Ceriotti, *Smooth, Exact Rotational Symmetrization for Deep Learning on Point Clouds*, Advances in Neural Information Processing Systems **36** (2023), conference paper; the supplied article is arXiv:2305.19302v3 (6 February 2024), including Appendices A–H.

[37] L. Vanduyfhuys *et al.*, *QuickFF: A Program for a Quick and Easy Derivation of Force Fields for Metal–Organic Frameworks from Ab Initio Input*, Journal of Computational Chemistry **36**, 1015–1027 (2015).

[38] G. J. Martyna, D. J. Tobias, and M. L. Klein, *Constant Pressure Molecular Dynamics Algorithms*, Journal of Chemical Physics **101**, 4177–4189 (1994), doi:10.1063/1.467468.

[39] W. Shinoda, M. Shiga, and M. Mikami, *Rapid estimation of elastic constants by molecular dynamics simulation under constant stress*, Physical Review B **69**, 134103 (2004), doi:10.1103/PhysRevB.69.134103.

[40] S. Nosé, *A unified formulation of the constant temperature molecular dynamics methods*, Journal of Chemical Physics **81**, 511–519 (1984), doi:10.1063/1.447334.

[41] W. G. Hoover, *Canonical dynamics: Equilibrium phase-space distributions*, Physical Review A **31**, 1695–1697 (1985), doi:10.1103/PhysRevA.31.1695.

[42] G. J. Martyna, M. L. Klein, and M. Tuckerman, *Nosé–Hoover Chains: The Canonical Ensemble via Continuous Dynamics*, Journal of Chemical Physics **97**, 2635–2643 (1992), doi:10.1063/1.463940.

[43] F. Bigi, S. Chong, M. Ceriotti, and F. Grasselli, *A prediction rigidity formalism for low-cost uncertainties in trained neural networks*, Machine Learning: Science and Technology **5**, 045018 (2024), doi:10.1088/2632-2153/ad805f.

[44] G. Imbalzano *et al.*, *Uncertainty estimation for molecular dynamics and sampling*, Journal of Chemical Physics **154**, 074102 (2021), doi:10.1063/5.0036522.

[45] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, *Automatic Differentiation in Machine Learning: A Survey*, Journal of Machine Learning Research **18**, 1–43 (2018), JMLR 18(153).

[46] A. Hill, G. Dalle, and A. Montoison, *An Illustrated Guide to Automatic Sparse Differentiation*, ICLR Blogposts (2025), online article.

[47] M. F. Langer, A. Hill, and M. Ceriotti, *Truncated automatic sparse differentiation for machine learning interatomic potentials*, arXiv:2609.20510 (2026), doi:10.48550/arXiv.2609.20510.

[48] F. Bigi *et al.*, *metatensor and metatomic: Foundational libraries for interoperable atomistic machine learning*, Journal of Chemical Physics **164**, 064113 (2026), doi:10.1063/5.0304911.

[49] S. Chorna, J. Lála, Q. Xu, R. K. Cersonsky, G. Fraux, and M. Ceriotti, *Chemiscope 1.0: interactive exploration of atomistic data from analysis to dissemination*, Journal of Open Source Software **11**(122), 10380 (2026), doi:10.21105/joss.10380.

A Transformers and attention

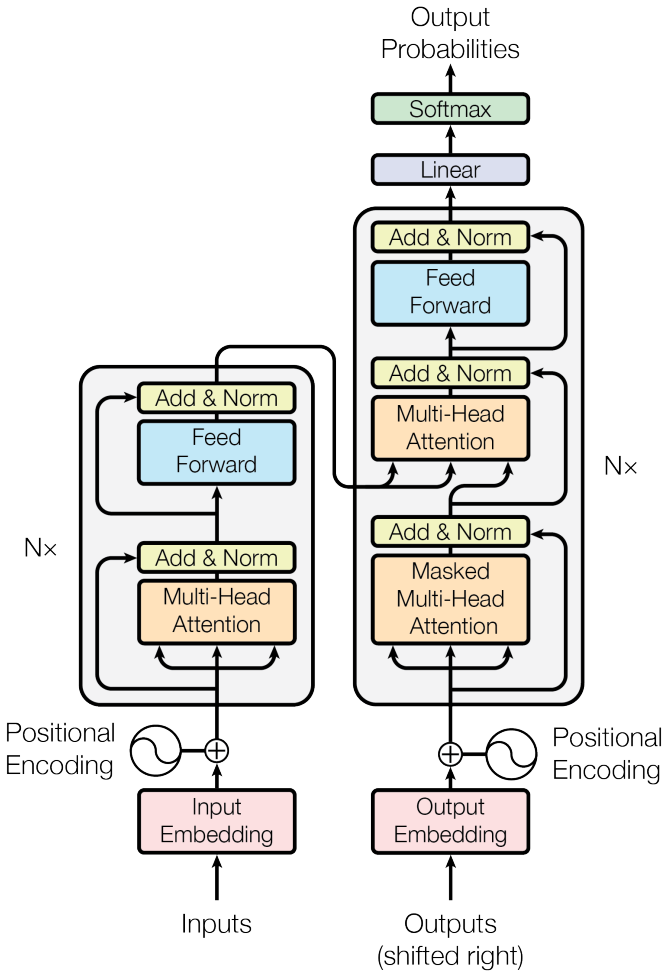


Figure 13: Transformer architecture developed in Ref. [30]. The attention mechanism is the part in gray. The part of the transformer on the left is called the encoder, and the part on the right is called the decoder. The encoder takes a sequence of tokens as input and produces a set of vectors, whose representations capture the meaning of the input tokens and their context. The decoder uses these vectors to generate an output sequence and guesses the next token in the sequence.

The transformer architecture has been revolutionary in the field of AI as a whole. It was initially developed for natural language processing (NLP) tasks, in particular for translation. But it has since been applied to a wide range of other domains, including computer vision, speech processing, code generation, protein folding. It is a state-of-the-art architecture that has been used to build some of the most powerful AI models, such as GPT-3 and GPT-4. It has also been applied to material science, in particular in MLIPs, where it has been used to predict the properties of materials and molecules.

A.1 Tokenization and embedding

The original transformer architecture [30] was designed for sentence translation. But it was then used to other NLP tasks, such as text classification and question answering. Therefore, the transformer architecture takes a text as input. However, a text is a sequence of strings, which is not numerical data and can't be directly processed by a neural network. To convert a sentence into a numerical representation, each word (or punctuation mark) in the sentence is first separated

into a token, and then each token is mapped to a vector representation, called an embedding. Each token embedding is a vector living in a high-dimensional space, and the embedding vectors are stacked together to form a matrix $\boldsymbol{X}$. The way to interpret these vectors is that each token is represented as a point in a high-dimensional space, and the distance between two points reflects the semantic similarity between the corresponding tokens. For instance, the tokens "cat" and "dog" are both animals, so their embeddings should be close to each other in the embedding space. In addition, the transformer architecture also uses positional encoding to encode the order of the tokens in the sequence. This is especially important for words that could have different meanings depending on their position in the sentence. For instance, the word "bank" could refer to a financial institution or the side of a river, and its meaning can be inferred from the surrounding words. So the embedding of the word "bank" could have different values depending on its position in the sentence, and the positional encoding helps to capture this information. These embeddings are something that has to be learned from the data, and they are usually initialized randomly and then updated during training. In the case of the PET architecture, the token embeddings are already explained in Section 2.5. So what is relevant for us is to understand the next step in the transformer architecture, which is the attention mechanism.

A.2 Attention mechanism

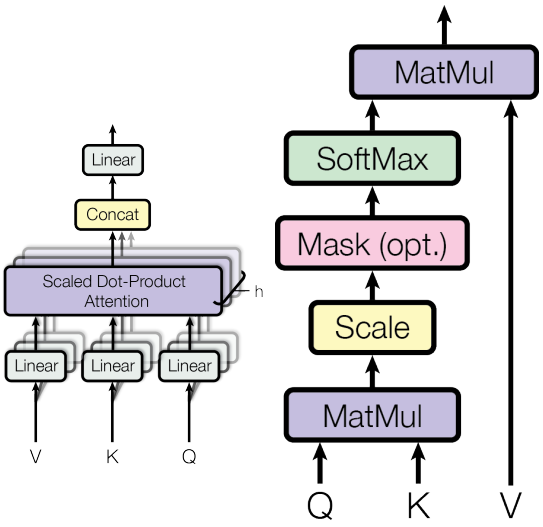


Figure 14: On the left, we have a multi-head attention mechanism, which is the orange part of the transformer architecture in Figure 13. On the right, we see the scaled dot-product attention mechanism, which is the blue part on the left figure. Pictures are taken from Ref. [30].

With our input matrix $\boldsymbol{X} \in \mathbb{R}^{n \times d}$, where n is the number of tokens and d is the embedding dimension, we compute the queries, keys, and values as follows:

$$\boldsymbol{Q} = \boldsymbol{X}\boldsymbol{W}_Q, \qquad \boldsymbol{K} = \boldsymbol{X}\boldsymbol{W}_K, \qquad \boldsymbol{V} = \boldsymbol{X}\boldsymbol{W}_V, \tag{A.1}$$

with $\boldsymbol{W}_Q, \boldsymbol{W}_K \in \mathbb{R}^{d \times d_k}$ and $\boldsymbol{W}_V \in \mathbb{R}^{d \times d_v}$. Thus $\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V} \in \mathbb{R}^{n \times d}$. These matrices are what the attention mechanism uses to compute the attention matrix $\boldsymbol{A} \in \mathbb{R}^{n \times n}$, which contains the attention weights for each token pair.

$$\boldsymbol{A} = \mathrm{softmax}_{\mathrm{row}}\left(\frac{\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}}{\sqrt{d_k}} + \boldsymbol{M}\right), \tag{A.2}$$

To better understand what $\boldsymbol{A}$ represents, we can look at its coefficients. Suppose we have a sentence "The animal didn't cross the street because it was too tired". The word "it" refers to "the animal" and not "the street", so the attention weight between "it" and "the animal" should be high, while the attention weight between "it" and "the street" should be low. The term $\boldsymbol{Q}\boldsymbol{K}^{\mathrm{T}}$ computes the dot product between the queries and keys, which measures the similarity between each token's query and every other token's key. The scaling factor $1/\sqrt{d_k}$ is used to prevent the dot products from growing too large. The $\boldsymbol{M}$ matrix is a mask that can be used to prevent certain tokens from attending to each other. For instance, if we are training the model to predict the next word in a sentence, we don't want the model to know the future words, otherwise it would defeat the purpose of predicting the next word, so we "mask" these words by setting their attention weights to zero. The softmax function is given by

$$\mathrm{softmax}_{\mathrm{row}}(\boldsymbol{Z})_{ab} = \frac{\exp(Z_{ab})}{\sum_{c=1}^{n}\exp(Z_{ac})}, \qquad a, b = 1, \ldots, n, \tag{A.3}$$

The softmax function normalizes the attention weights so that they sum to one across each row, which means that for each token, the attention weights represent a probability distribution over all other tokens. We add large negative values to the masked positions in $\boldsymbol{M}$ so that their softmax output is effectively zero. After computing $\boldsymbol{A}$, we can compute the output of the attention mechanism as a weighted sum of the values:

$$\boldsymbol{Y} = \boldsymbol{A}\boldsymbol{V}, \tag{A.4}$$

where $\boldsymbol{Y} \in \mathbb{R}^{n \times d_v}$ is the output of the attention mechanism. Each of the W_Q, W_K, and W_V are weight matrices that are fine-tuned during training. In the PET architecture, masking is not needed because there is no sequential ordering of the tokens, and all tokens can attend to each other.

A.3 Multi-head attention

We can extend the attention mechanism by dividing the input tokens $\boldsymbol{X}$ into subsets $\boldsymbol{X}_1, \ldots, \boldsymbol{X}_H$, such that $d_v = d/H$, and constructing attention matrices $\boldsymbol{A}_1, \ldots, \boldsymbol{A}_H$ for each subset, this is called multi-head attention. We can then concatenate the outputs of each head to form the final output:

$$\mathrm{MHA}(\boldsymbol{X}) = \mathrm{Concat}(\boldsymbol{Y}_1, \ldots, \boldsymbol{Y}_H)\boldsymbol{W}_O, \qquad \boldsymbol{W}_O \in \mathbb{R}^{Hd_v \times d}. \tag{A.5}$$

The two main advantages of using multi-head attention instead of simple attention are that it allows to make the process more parallelizable, and it allows the model to learn more complex relationships between the tokens, since it also adds more parameters to the model. A common choice is $d_k = d_v = d/H$ for $d = 256$ and eight heads this gives 32 components per head. These dimensions describe attention projections, rather than the width of a later energy readout head.

B Sparse automatic differentiation

Automatic sparse differentiation (ASD) is a method for efficiently computing derivatives of neural networks. It makes use of the sparsity of the Jacobian (or Hessian) matrix to reduce the memory usage of these matrices, by storing only the non-zero elements. The sparsity pattern of these matrices also allows us to compute the derivatives more efficiently, by avoiding unnecessary computations of zero elements. This method provides an exact computation of the derivatives, but we can make some approximations to further reduce the computations. The approximation is explained in the Hessian truncation section.

B.1 The Jacobian, JVPs, and VJPs

For a function $\boldsymbol{f} : \mathbb{R}^m \to \mathbb{R}^n$, the Jacobian matrix is defined as

$$\boldsymbol{J}(\boldsymbol{x}) = \frac{\partial \boldsymbol{f}}{\partial \boldsymbol{x}}(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_n}{\partial x_m} & \cdots & \frac{\partial f_n}{\partial x_m} \end{pmatrix} \in \mathbb{R}^{n \times m} \tag{B.1}$$

We can compute the directional derivative of $\boldsymbol{f}$ at $\boldsymbol{x}$ along a vector $\boldsymbol{v} \in \mathbb{R}^m$ as

$$\frac{\mathrm{d}}{\mathrm{d}\epsilon} \boldsymbol{f}(\boldsymbol{x} + \epsilon \boldsymbol{v}) \bigg|_{\epsilon=0} = \boldsymbol{J}(\boldsymbol{x}) \boldsymbol{v}. \tag{B.2}$$

This is what we call a *Jacobian–vector product* (JVP). It is the instantaneous rate of change of $\boldsymbol{f}$ along the direction $\boldsymbol{v}$. Since the Jacobian at a point $\boldsymbol{x}$ is a linear map, the directional derivative is linear in $\boldsymbol{v}$. This means that we can recover all the columns of the Jacobian by evaluating the JVPs along the standard basis vectors of $\mathbb{R}^m$: $\boldsymbol{e}_1, \ldots, \boldsymbol{e}_m$. Let's illustrate this with a simple example. Consider the function $\boldsymbol{f} : \mathbb{R}^4 \to \mathbb{R}^3$

$$\boldsymbol{f}(\boldsymbol{x}) = \begin{pmatrix} x_1^2 + x_2 \\ \sin(x_2) + x_3^2 \\ e^{x_4} \end{pmatrix}, \tag{B.3}$$

The Jacobian is

$$\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \\ 0 & \cos(x_2) & 2x_3 & 0 \\ 0 & 0 & 0 & e^{x_4} \end{pmatrix}. \tag{B.4}$$

If we try evaluating the JVPs along the standard basis vectors, we get

$$\boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_1 = \begin{pmatrix} 2x_1 \\ 0 \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_2 = \begin{pmatrix} 1 \\ \cos(x_2) \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_3 = \begin{pmatrix} 0 \\ 2x_3 \\ 0 \end{pmatrix}, \quad \boldsymbol{J}(\boldsymbol{x})\boldsymbol{e}_4 = \begin{pmatrix} 0 \\ 0 \\ e^{x_4} \end{pmatrix}. \tag{B.5}$$

Which exactly recovers the columns of the Jacobian in Eq. (B.4). However, the same result could have been obtained if instead of evaluating the JVPs along the standard basis vectors of $\mathbb{R}^m$, we had used the standard basis vectors of $\mathbb{R}^n$ and evaluated the vector-Jacobian products

(VJP) along the standard basis vectors of $\mathbb{R}^n$.

$$\boldsymbol{e}_1^{\mathrm{T}}\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \end{pmatrix}, \quad \boldsymbol{e}_2^{\mathrm{T}}\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 0 & \cos(x_2) & 2x_3 & 0 \end{pmatrix}, \quad \boldsymbol{e}_3^{\mathrm{T}}\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 0 & 0 & 0 & e^{x_4} \end{pmatrix}. \tag{B.6}$$

This time, we recover the rows of the Jacobian instead of the columns. The advantage of using VJPs here instead of JVPs is that if $n \ll m$, which means we can recover the Jacobian with fewer evaluations. This is particularly useful when dealing with neural networks, where the number of outputs is usually much smaller than the number of inputs. In particular, in the case MLIPs, the number of outputs is only one, the energy of the system, while the number of inputs depends on the number of atoms in the system and can be much larger. For a composition of functions $\boldsymbol{f} = \boldsymbol{h} \circ \boldsymbol{g}$, the chain rule gives the Jacobian as

$$\boldsymbol{J}_f(\boldsymbol{x})\boldsymbol{v} = \boldsymbol{J}_h(\boldsymbol{g}(\boldsymbol{x}))\big[\boldsymbol{J}_g(\boldsymbol{x})\boldsymbol{v}\big]. \tag{B.7}$$

B.2 Recovering multiple columns with a single evaluation

In some cases, we can recover multiple columns of the Jacobian with a single VJP evaluation. For instance, if we evaluate the VJP along the vector $\boldsymbol{v} = \boldsymbol{e}_1 + \boldsymbol{e}_3 + \boldsymbol{e}_4$, we get

$$\boldsymbol{J}(\boldsymbol{x})\boldsymbol{v} = \begin{pmatrix} 2x_1 \\ 2x_3 \\ e^{x_4} \end{pmatrix}. \tag{B.8}$$

The elements of this vector are the only nonzero entries in the first, third, and fourth columns of the Jacobian. By exploiting the sparsity of the Jacobian, we can recover multiple columns with a single evaluation. This is possible because column 1 has zeros in rows 2 and 3, column 3 has zeros in rows 1 and 3, and column 4 has zeros in rows 1 and 2. In other words, the nonzero entries of these columns do not overlap in any row. The nonoverlapping property is what allows us to recover multiple columns with a single evaluation. The last remaining column can be recovered with a second evaluation along $\boldsymbol{e}_2$. Thus, we can recover the entire Jacobian with only two evaluations instead of four as previously done.

B.3 Graph coloring

To better understand which columns can be grouped together, we can represent the sparsity pattern as a graph. Each column in $\boldsymbol{J}$ corresponds to a vertex in the graph and an edge connects two vertices if they have one or more nonzero entries in the same row. In our example, columns 1 and 2 have a nonzero entry in row 1, so they are connected by an edge. Same for columns 2 and 3, which have a nonzero entry in row 2. The set of vertices and edges forms a graph $G = (V, E)$, where $V = \{1, 2, 3, 4\}$ is the set of vertices (columns of $\boldsymbol{J}$) and $E = \{\{1, 2\}, \{2, 3\}\}$ is the set of edges (connections between columns). We can represent a graph with a diagram, where vertices are drawn as circles and edges as lines connecting them.

The next step is to assign a color to each vertex in the graph such that no two adjacent vertices share the same color. This process is called *graph coloring*. The goal is to find a coloring that uses the smallest number of colors possible $\chi(G)$, which is called the *chromatic number* of

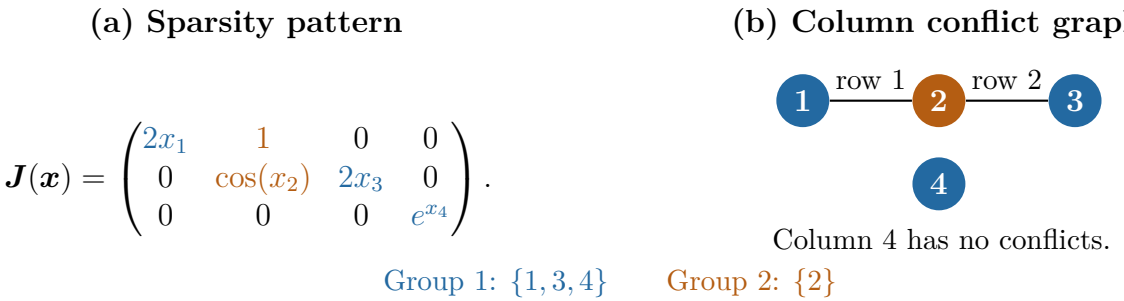


Figure 15: The same four-input example viewed as a pattern and as a graph. Colored coefficients mark possible nonzero entries. Graph edges record conflicts between columns, with the responsible row written beside each edge. Blue columns 1, 3, and 4 have no mutual conflicts and share one seed; orange column 2 uses another. A color denotes a group of columns.

the graph. The number of colors in the graph corresponds to the number of JVP evaluations needed to recover all columns of $\boldsymbol{J}$. The columns of the same color have no conflicts and can be computed together in a single JVP evaluation. In our example, $\chi(G) = 2$, as shown in Fig. 15. Columns 1, 3, and 4 can be assigned the same color (blue), while column 2 can be assigned another color (orange). As we saw previously, we could recover columns 1, 3, and 4 with a single JVP evaluation along $\boldsymbol{e}_1 + \boldsymbol{e}_3 + \boldsymbol{e}_4$, and column 2 with another JVP evaluation along $\boldsymbol{e}_2$. Which is consistent with the coloring we found. For this simple example, the chromatic number is easy to find by inspection. But in general, finding the chromatic number of a graph is an NP-hard problem. Therefore, we need to use certain algorithms to find a coloring that uses a small number of colors, even if it is not the minimum possible.

B.4 Decompression

After finding a coloring, we can evaluate the JVPs along the vectors belonging to each vector. We can group them in a matrix $\boldsymbol{S} \in \mathbb{R}^{m \times c}$ called the *seed matrix*. For our example, the seed matrix is

$$\boldsymbol{S} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \end{pmatrix}, \tag{B.9}$$

We can then get a compressed Hessian by computing the product $\boldsymbol{J}(\boldsymbol{x})\boldsymbol{S}$. For our example, we get

$$\boldsymbol{J}(\boldsymbol{x})\boldsymbol{S} = \begin{pmatrix} 2x_1 & 1 \\ 2x_3 & \cos(x_2) \\ e^{x_4} & 0 \end{pmatrix}. \tag{B.10}$$

This compressed matrix contains all the information needed to recover the original Jacobian. Instead of storing a full 3×4 Jacobian, we only need to store a 3×2 compressed matrix. To recover the original $\boldsymbol{J}$, we can use the coloring information to unpack the compressed matrix. The first compressed column supplies the blue entries, and the second supplies the orange entries:

$$
\begin{aligned}
J_{11} = Y_{11} &= 2x_1, & J_{12} = Y_{12} &= 1, \\
J_{23} = Y_{21} &= 2x_3, & J_{22} = Y_{22} &= \cos(x_2), \\
J_{34} = Y_{31} &= e^{x_4}.
\end{aligned} \tag{B.11}
$$

The sparsity pattern places these values in their original columns and sets all other entries to zero:

$$
\boldsymbol{J}(\boldsymbol{x}) = \begin{pmatrix} 2x_1 & 1 & 0 & 0 \\ 0 & \cos(x_2) & 2x_3 & 0 \\ 0 & 0 & 0 & e^{x_4} \end{pmatrix}. \tag{B.12}
$$

Recovering $\boldsymbol{J}$ from $\boldsymbol{J}(\boldsymbol{x})\boldsymbol{S}$ is called *decompression*.

B.5 The Hessian and HVPs

For a function $f : \mathbb{R}^m \to \mathbb{R}$, the Hessian matrix is defined as

$$
\boldsymbol{H}(\boldsymbol{x}) = \begin{pmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_m} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_m \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_m^2} \end{pmatrix} \in \mathbb{R}^{m \times m}. \tag{B.13}
$$

The methods we have discussed for the Jacobian can also be applied to the Hessian. In fact, it can be applied to any sparse matrix, regardless of whether it is a Jacobian or Hessian. Therefore, the same coloring and decompression techniques can be used to recover the Hessian with fewer evaluations. Since the Hessian is symmetric, we can exploit this property to further reduce the number of evaluations needed. Furthermore, computing the Hessian-vector product (HVP) is equivalent to computing the vector-Hessian product (VHP) due to the symmetry of the Hessian. For instance, suppose we have a function $f : \mathbb{R}^4 \to \mathbb{R}$ defined as

$$
f(\boldsymbol{x}) = x_1^2 + x_1 x_2 + x_3^2 + x_4^2, \tag{B.14}
$$

Its Hessian is given by

$$
\boldsymbol{H}(\boldsymbol{x}) = \begin{pmatrix} 2 & 1 & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 2 \end{pmatrix}. \tag{B.15}
$$

Using the same graph coloring technique as before, we can find a coloring for the Hessian sparsity pattern. Columns 1 and 2 are the only ones that have a nonzero entry in the same row, so they cannot be assigned the same color. So $\chi(G) = 2$ and we can again assign blue to columns 1, 3, and 4 and orange to column 2. Colors follow columns, so the symmetric entries H_{12} and H_{21} have different colors despite having the same value. Therefore, we can get the seed matrix $\boldsymbol{S}$ as

$$\boldsymbol{S} = \begin{pmatrix} 1 & 0 \\ 0 & 1 \\ 1 & 0 \\ 1 & 0 \end{pmatrix}. \tag{B.16}$$

Then we can get the compressed Hessian by computing the product $\boldsymbol{H}(\boldsymbol{x})\boldsymbol{S}$

$$\boldsymbol{H}(\boldsymbol{x})\boldsymbol{S} = \begin{pmatrix} 2 & 1 \\ 1 & 0 \\ 2 & 0 \\ 2 & 0 \end{pmatrix}. \tag{B.17}$$

To recover the original Hessian, we can use the coloring information to unpack the compressed matrix

$$\begin{aligned} H_{11} = Y_{11} = 2, \qquad & H_{12} = Y_{12} = 1, \\ H_{21} = Y_{21} = 1, \qquad & H_{33} = Y_{31} = 2, \\ H_{44} = Y_{41} = 2. & \end{aligned} \tag{B.18}$$

All other coefficients are structural zeros.

B.6 ASD in PET

Since the potential energy computed by PET is the sum of the contributions from each atomic environment A_i, an atom i can only interact with its neighbors in A_i. Therefore, when computing the second derivatives of the potential, only atoms inside A_i can contribute to the Hessian entries $H_{i\alpha,j\beta}$, the other entries are structural zeros. The Hessian can be written as blocks of 3×3 matrices, where each block corresponds to the second derivatives of the potential with respect to the coordinates of two atoms i and j:

$$H_{i\alpha,j\beta} = \sum_t \frac{\partial^2 \varepsilon_t}{\partial R_{i\alpha} \partial R_{j\beta}}. \tag{B.19}$$

We can represent the pairs of atoms that interact with each other as a graph, where each atom is a vertex and an edge connects two atoms if they are neighbors in the same atomic environment. A compact way of visualizing these interactions is by using an adjacency matrix $\boldsymbol{A}$, where $A_{ij} = 1$ if atoms i and j are connected, and 0 otherwise. It is important to construct this matrix, because by doing so we can identify the nonzero entries of the Hessian and thus use the graph coloring technique to detect the sparsity pattern of the Hessian, without having to compute the Hessian explicitly. With this graph, we can identify paths between different atoms, even if they are not directly connected. This is achieved by traversing intermediate nodes, which allows us to find all pairs of atoms that are connected by a path of length k or less. The number of nodes to get from one atom to another is called a hop. We can determine the k-hop pattern by computing the powers of the adjacency matrix $\boldsymbol{A}$.

$$\boldsymbol{A}^{(k)} = \bigvee_{i=1}^{k} \boldsymbol{A}^i \tag{B.20}$$

The coefficients $\boldsymbol{A}_{ij}^{(k)}$ are 1 if atoms i and j are connected by a path of length k or less, and 0 otherwise. For instance, if $(\boldsymbol{A}^{(2)})_{ij} = 1$, atoms i and j are connected by a path of length 2 or less.

B.7 Hessian truncation

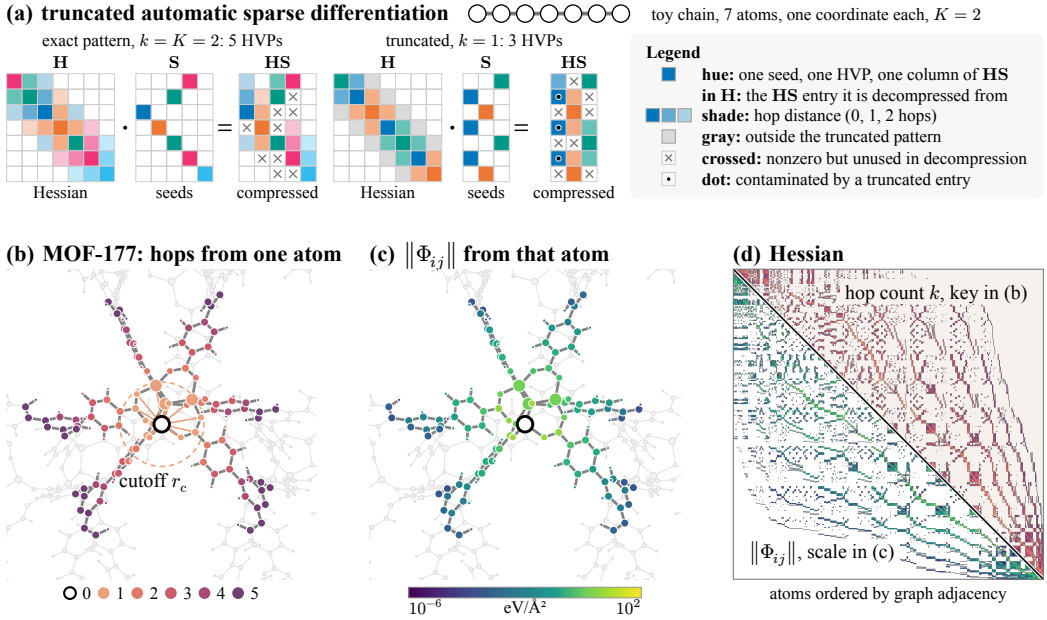


Figure 16: Truncation error in the Hessian. The blue blocks are retained, and the orange blocks are discarded. The retained blocks are contaminated by the omitted couplings, which are still present in the energy function. Figure adapted from [47].

The number of hops K needed to recover the full Hessian is given by $K = 2L + 1$, where L is the number message-passing layers in the PET architecture. For instance, PET-MAD-1.5-S has $L = 3$ message-passing layers, so we need $K = 7$ hops to recover the full Hessian. Therefore, $\boldsymbol{A}^{(7)}$ tell us which entries are nonzero and we can use it to find the coloring of the Hessian. The higher K is, the more nonzero entries we have in the Hessian, and the more evaluations we need to recover it. If we truncate the Hessian to a smaller number of hops $k < K$, $\boldsymbol{A}^{(k)}$ becomes sparser and fewer HVPs are needed to recover the truncated Hessian. It has been shown in [47] that this is a reasonable approximation, since the Hessian entries decay with distance and it allows for a more efficient computation. We can see this in Fig. 16 on plot (b), where we have the atoms around the central atom, each having a color corresponding to the number of hops from the central atom. We see that all the one hop neighbors are located inside the cutoff radius. The further away we are from the central atom, the higher the number of hops. In the plot (c), we have the frobenius norm of the Hessian blocks for the previous example between the central atom and its neighbors. It is clear from this plot that the norm decays with the number of hops, thus justifying the truncation of the Hessian.

C Temperature and pressure control

C.1 Thermostat

A thermostat exchanges energy with the particles to sample a target temperature T. The equipartition theorem states that each independent quadratic degree of freedom contributes $k_{\mathrm{B}}T/2$ to the mean energy [7]:

$$\left\langle \frac{ax^2}{2} \right\rangle = \frac{1}{2}k_{\mathrm{B}}T, \qquad a > 0. \tag{C.1}$$

For f active atomic momentum degrees of freedom, this gives

$$K = \sum_{i=1}^{N} \frac{|\boldsymbol{P}_i|^2}{2M_i}, \qquad \langle K \rangle = \frac{f}{2}k_{\mathrm{B}}T. \tag{C.2}$$

Thus

$$T = \frac{2\langle K \rangle}{f k_{\mathrm{B}}}, \qquad T_{\mathrm{kin}} = \frac{2K}{f k_{\mathrm{B}}}. \tag{C.3}$$

Here $f = 3N$ for unconstrained momenta, or $3N - 3$ when total linear momentum is removed. The instantaneous T_{kin} fluctuates around the target.

Nosé introduces a positive generalized coordinate s and its conjugate momentum p_s [40]. At fixed cell, the extended Hamiltonian is

$$\mathcal{H}_{\mathrm{N}}(\boldsymbol{R}^N, s; \boldsymbol{\pi}^N, p_s) = \sum_{i=1}^{N} \frac{|\boldsymbol{\pi}_i|^2}{2M_i s^2} + U(\boldsymbol{R}^N) + \frac{p_s^2}{2Q} + f k_{\mathrm{B}}T \ln s. \tag{C.4}$$

The generalized coordinates are $(\boldsymbol{R}^N, s)$ and the canonical momenta are $(\boldsymbol{\pi}^N, p_s)$; Q is the thermostat inertia. The coefficient f is chosen for canonical sampling in physical time. Hamilton's equations in Nosé's auxiliary time τ are

$$\frac{\mathrm{d}\boldsymbol{R}_i}{\mathrm{d}\tau} = \frac{\partial \mathcal{H}_{\mathrm{N}}}{\partial \boldsymbol{\pi}_i} = \frac{\boldsymbol{\pi}_i}{M_i s^2}, \tag{C.5}$$

$$\frac{\mathrm{d}\boldsymbol{\pi}_i}{\mathrm{d}\tau} = -\frac{\partial \mathcal{H}_{\mathrm{N}}}{\partial \boldsymbol{R}_i} = \boldsymbol{F}_i, \qquad \boldsymbol{F}_i = -\nabla_{\boldsymbol{R}_i} U, \tag{C.6}$$

$$\frac{\mathrm{d}s}{\mathrm{d}\tau} = \frac{\partial \mathcal{H}_{\mathrm{N}}}{\partial p_s} = \frac{p_s}{Q}, \tag{C.7}$$

$$\frac{\mathrm{d}p_s}{\mathrm{d}\tau} = -\frac{\partial \mathcal{H}_{\mathrm{N}}}{\partial s} = \sum_i \frac{|\boldsymbol{\pi}_i|^2}{M_i s^3} - \frac{f k_{\mathrm{B}}T}{s}. \tag{C.8}$$

Using physical time $\mathrm{d}t = \mathrm{d}\tau/s$, physical momenta $\boldsymbol{P}_i = \boldsymbol{\pi}_i/s$, and $\eta = \ln s$ gives the Nosé–Hoover equations [41]:

$$\dot{\boldsymbol{R}}_i = \frac{\boldsymbol{P}_i}{M_i}, \qquad \dot{\boldsymbol{P}}_i = \boldsymbol{F}_i - \dot{\eta}\boldsymbol{P}_i, \qquad Q\ddot{\eta} = 2K - f k_{\mathrm{B}}T. \tag{C.9}$$

Dots now denote physical-time derivatives. The thermostat removes excess kinetic energy by increasing the friction $\dot{\eta}$, and reverses this feedback when the kinetic energy is below its target.

The project uses a chain of three thermostat variables to improve sampling [42]. Introduce coordinates $\boldsymbol{\eta} = (\eta_1, \eta_2, \eta_3)$ and auxiliary momenta $p_{\eta_j} = Q_j \dot{\eta}_j$. Its conserved extended energy, often called the chain Hamiltonian, is

$$\begin{aligned} &\mathcal{H}_{\mathrm{NHC}}(\boldsymbol{R}^N, \eta_1, \eta_2, \eta_3; \boldsymbol{P}^N, p_{\eta_1}, p_{\eta_2}, p_{\eta_3}) \\ &\quad = K + U(\boldsymbol{R}^N) + \sum_{j=1}^{3} \frac{p_{\eta_j}^2}{2Q_j} + k_{\mathrm{B}}T(f\eta_1 + \eta_2 + \eta_3). \end{aligned} \tag{C.10}$$

Keeping only the first thermostat recovers the single Nosé–Hoover extended energy. In these physical-time variables the dynamics is noncanonical: ordinary Hamilton's equations applied to this energy alone would omit the friction couplings. The corresponding generalized equations are

$$\dot{\boldsymbol{R}}_i = \frac{\boldsymbol{P}_i}{M_i}, \qquad \dot{\boldsymbol{P}}_i = \boldsymbol{F}_i - \frac{p_{\eta_1}}{Q_1}\boldsymbol{P}_i, \tag{C.11}$$

$$\dot{\eta}_j = \frac{p_{\eta_j}}{Q_j}, \qquad j = 1, 2, 3, \tag{C.12}$$

$$\dot{p}_{\eta_1} = 2K - fk_{\mathrm{B}}T - \frac{p_{\eta_2}}{Q_2}p_{\eta_1}, \tag{C.13}$$

$$\dot{p}_{\eta_2} = \frac{p_{\eta_1}^2}{Q_1} - k_{\mathrm{B}}T - \frac{p_{\eta_3}}{Q_3}p_{\eta_2}, \tag{C.14}$$

$$\dot{p}_{\eta_3} = \frac{p_{\eta_2}^2}{Q_2} - k_{\mathrm{B}}T. \tag{C.15}$$

Each additional thermostat controls the kinetic energy of the preceding one. Equivalently, $\xi_j = \dot{\eta}_j = p_{\eta_j}/Q_j$ are the friction variables.

C.2 Barostat

A barostat on the other hand exchanges work with the simulation cell to keep the system at a target pressure P_{ext}. The barostat adjusts the simulation cell size and shape to maintain the desired pressure. This project uses the flexible-cell formulation of Shinoda, Shiga, and Mikami [39], combining Martyna–Tobias–Klein (MTK) pressure control [38] with Parrinello–Rahman strain dynamics, as implemented in LAMMPS `fix npt` [17]. The instantaneous pressure includes kinetic and configurational terms:

$$\boldsymbol{\Pi}_{\mathrm{int}} = \frac{1}{V}\left[\sum_i \frac{\boldsymbol{P}_i \otimes \boldsymbol{P}_i}{m_i} + \boldsymbol{W}_{\mathrm{vir}}\right], \qquad P_{\mathrm{int}} = \frac{1}{3}\operatorname{tr}\boldsymbol{\Pi}_{\mathrm{int}}, \tag{C.16}$$

where $\boldsymbol{W}_{\mathrm{vir}}$ is the configurational virial tensor. The feedback is illustrated by the isotropic MTK equations. Let $\nu = \dot{V}/(3V)$ be the linear expansion rate, W_b the fictitious barostat inertia, and ξ_b the first friction variable of the chain thermostatting the barostat. For an imposed pressure P_{ext},

$$\dot{\boldsymbol{R}}_i = \frac{\boldsymbol{P}_i}{m_i} + \nu \boldsymbol{R}_i, \qquad \dot{V} = 3\nu V, \tag{C.17}$$

$$\dot{\boldsymbol{P}}_i = \boldsymbol{F}_i - \left[\xi_1 + \left(1 + \frac{3}{f}\right)\nu\right] \boldsymbol{P}_i, \tag{C.18}$$

$$W_b \dot{\nu} = 3V(P_{\text{int}} - P_{\text{ext}}) + \frac{6K}{f} - \xi_b W_b \nu. \tag{C.19}$$

The pressure difference accelerates the cell outward or inward; cell motion rescales positions and momenta and exchanges mechanical work with the atoms. The $3/f$ momentum correction and $6K/f$ barostat term ensure the intended NPT statistical measure [38].